

UNIwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych
Instytut Informatyki



Oleksii Nawrocki
oa131400

Informatyka

Sprawozdanie z ćwiczeń laboratoryjnych z przedmiotu
Biometryczne Systemy Zabezpieczeń

Sprawozdania
Grupa: 02
Zespół nr: XX

Prowadzący
dr Zbigniew Gomółka

Rzeszów 2025

Spis treści

1. Laboratorium 1	7
1.1. Cel ćwiczenia	7
1.2. Wstęp teoretyczny	7
1.3. Opis wykorzystywanego oprogramowania i narzędzi	7
1.4. Zadania do wykonania samodzielnego	8
1.4.1. Zadanie 1	8
1.4.2. Zadanie 2	11
1.4.3. Zadanie 3	13
1.4.4. Zadanie 4	15
1.4.5. Zadanie 5	17
1.5. Wnioski	18
2. Laboratorium 2	20
2.1. Cel ćwiczenia	20
2.2. Wstęp teoretyczny	20
2.3. Opis wykorzystywanego oprogramowania i narzędzi	21
2.4. Zadania do wykonania samodzielnego	22
2.4.1. Zadanie 1	22
2.5. Wnioski	26
3. Laboratorium 3	27
3.1. Cel ćwiczenia	27
3.2. Wstęp teoretyczny	27
3.2.1. Szkieletyzacja obrazów binarnych	27
3.2.2. Algorytm Chin-Van-Stover-Iverson	28
3.2.3. Szkieletyzacja obrazów tonowanych	28
3.3. Zadania do wykonania samodzielnego	29
3.3.1. Zadanie 1	29
3.4. Wnioski	32
4. Laboratorium 4	34
4.1. Cel ćwiczenia	34
4.2. Wstęp teoretyczny	34
4.2.1. Transformata Hougha	34
4.3. Opis wykorzystywanego oprogramowania i narzędzi	36
4.4. Zadania do wykonania samodzielnego	36
4.4.1. Zadanie 1	36
4.4.2. Zadanie 2	38
4.5. Wnioski	40

5. Laboratorium 5	41
5.1. Cel ćwiczenia	41
5.2. Wstęp teoretyczny	41
5.3. Opis wykorzystywanego oprogramowania i narzędzi	42
5.4. Zadania do wykonania samodzielnego	42
5.4.1. Zadanie 1	42
5.4.2. Zadanie 2	45
5.4.3. Zadanie 3	47
5.5. Wnioski	50
5.5.1. Zadanie 1 – Szereg Fouriera dla sygnału okresowego	50
5.5.2. Zadanie 2 – Filtracja obrazu tęczy (2D FFT)	50
5.5.3. Zadanie 3 – Filtracja obrazu daktyloskopijnego (odcisk palca)	50
6. Laboratorium 6	51
6.1. Cel ćwiczenia	51
6.2. Wstęp teoretyczny	51
6.3. Opis wykorzystywanego oprogramowania i narzędzi	53
6.4. Zadania do wykonania samodzielnego	53
6.4.1. Zadanie 1	53
6.4.2. Zadanie 2	54
6.5. Wnioski	58
Bibliografia	60
Spis rysunków	61
Spis tabel	62
Spis listingów	63
Oświadczenie studenta o samodzielności pracy	64

1. Laboratorium 1

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	WPROWADZENIE DO BIBLIOTEKI MATLAB IMAGE PROCESSING ORAZ HISTOGRAMOWE METODY SEGMENTACJI OBRAZÓW.
Ćwiczenie nr:	01
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	23.04.2025
Data oddania ćwiczenia	13.05.2025

1.1. Cel ćwiczenia

Celem laboratorium jest zapoznanie się z podstawowymi funkcjami biblioteki Image Processing Toolbox w MATLAB-ie oraz zastosowanie metod segmentacji obrazów opartych na analizie histogramu. Należy zapoznać się z materiałami i przykładami zawartymi w niniejszych materiałach oraz zaproponować rozwiązania dla zadań do samodzielnego rozwiązania. Rozwiązania należy umieścić w sprawozdaniu, zgodnie z wymaganiami.

1.2. Wstęp teoretyczny

Histogram obrazu to funkcja przedstawiająca rozkład wartości pikseli w obrazie. Może być wykorzystywany do:

- Poprawy kontrastu obrazu (rozciąganie histogramu, wyrównywanie histogramu),
- Segmentacji obrazu na podstawie progów jasności (np. metoda Otsu),
- Analizy jakości obrazu, w tym identyfikacji prześwietlonych lub niedoświetlonych obszarów.

Histogram w obrazie monochromatycznym można obliczyć za pomocą funkcji $h(k)$, określającej liczbę pikseli o poziomie jasności k :

$$h(k) = \sum_{(x,y) \in \Omega} \delta(I(x,y) - k), \quad (1.1)$$

gdzie δ to funkcja Kroneckera, a Ω oznacza zbiór pikseli w obrazie.

1.3. Opis wykorzystywanego oprogramowania i narzędzi

Do przeprowadzenia eksperymentu wykorzystano środowisko MATLAB oraz bibliotekę Image Processing Toolbox. Kluczowe funkcje wykorzystane w analizie to:

- `imhist` - obliczanie histogramu obrazu,
- `imadjust` - rozciąganie histogramu,

- `histeq` - wyrównywanie histogramu,
- `graythresh` oraz `imbinarize` - segmentacja metodą Otsu.

1.4. Zadania do wykonania samodzielnego

1.4.1. Zadanie 1

1.4.1.1. Treść

Konwersja obrazu na różne formaty przy użyciu MATLAB i Image Processing Toolbox:

Użyj funkcji dostępnych w bibliotece IPT w MATLAB-ie do przekształcenia obrazu. Na Rys. 9 przedstawiono schemat przepływu i konwersji różnych typów obrazów w MATLAB-ie, z wykorzystaniem funkcji dostępnych w IPT.

1.4.1.2. Pytania do analizy

- Jak zmienia się zakres wartości pikseli w kolejnych przekształceniach?
 - RGB: 3 kanały (R, G, B), każdy z zakresem [0, 255]
 - Monochromatyczny (gray): pojedynczy kanał, zakres [0, 255]
 - Binarny: dwa poziomy – 0 (czarny) i 1 (biały)
 - Indeksowany: każdy piksel to indeks w tablicy kolorów (zazwyczaj 0–255 przy 256 kolorach)
- Jaka jest różnica w zajmowanej pamięci między różnymi formatami obrazów?
 - **RGB**: największy rozmiar – 3 bajty na piksel (8 bitów × 3 kanały).
 - **Gray**: 1 bajt na piksel.
 - **Binarny**: 1 bit na piksel (oszczędność miejsca).
 - **Indeksowany**: 1 bajt na piksel + osobna tablica kolorów (colormap).
- Jakie mogą być zastosowania obrazów indeksowanych i binarnych w praktyce?
 - Indeksowane:
 - * kompresja obrazów do GIF.
 - * aplikacje z ograniczoną paletą barw (np. grafika retro, przeglądarki).
 - Binarne:
 - * analiza obiektów (segmentacja, maski).
 - * OCR (rozpoznawanie tekstu).
 - * wykrywanie krawędzi, konturów.

1.4.1.3. Przykładowy program

Listing 1.1. Wczytanie obrazu i stworzenie konwersji

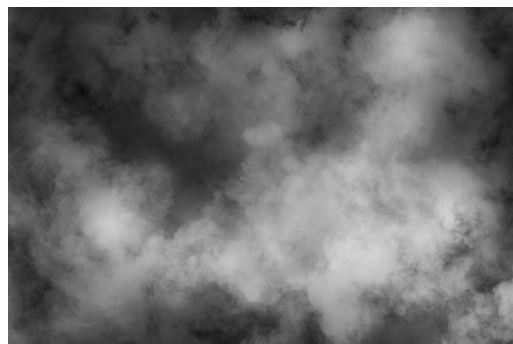
```
1 % Wczytaj obraz RGB
2 rgbImage = imread('colors.jpg'); % użyj dowolnego obrazu RGB
3 imshow(rgbImage);
4 title('Obraz oryginalny RGB');
```

```
5
6 % 1. Konwersja do obrazu monochromatycznego (odcienie szarości)
7 grayImage = rgb2gray(rgbImage);
8 figure, imshow(grayImage);
9 title('Obraz monochromatyczny');
10
11 % 2. Binaryzacja obrazu (konwersja do obrazu binarnego)
12 binaryImage = imbinarize(grayImage);
13 figure, imshow(binaryImage);
14 title('Obraz binarny');
15
16 % 3. Konwersja do obrazu indeksowanego (indeks + mapa kolorów)
17 [indexedImage, colormap] = rgb2ind(rgbImage, 256); % 256 kolorów
18 figure, imshow(indexedImage, colormap);
19 title('Obraz indeksowany');
20
21 % 4. Wyodrębnienie kanałów RGB
22 redChannel = rgbImage(:, :, 1);
23 greenChannel = rgbImage(:, :, 2);
24 blueChannel = rgbImage(:, :, 3);
25
26 figure,
27 subplot(1,3,1), imshow(redChannel), title('Kanał czerwony');
28 subplot(1,3,2), imshow(greenChannel), title('Kanał zielony');
29 subplot(1,3,3), imshow(blueChannel), title('Kanał niebieski');
30
31 % 5. Zapis do plików
32 imwrite(rgbImage, 'zadanielimages/obraz_rgb.png');
33 imwrite(grayImage, 'zadanielimages/obraz_szary.png');
34 imwrite(binaryImage, 'zadanielimages/obraz_binarny.png');
35 imwrite(indexedImage, colormap, 'zadanielimages/obraz_indeksowany.png');
36
37 % Zapis kanałów RGB
38 imwrite(redChannel, 'zadanielimages/kanal_czerwony.png');
39 imwrite(greenChannel, 'zadanielimages/kanal_zielony.png');
40 imwrite(blueChannel, 'zadanielimages/kanal_niebieski.png');
```

1.4.1.4. Obrazy



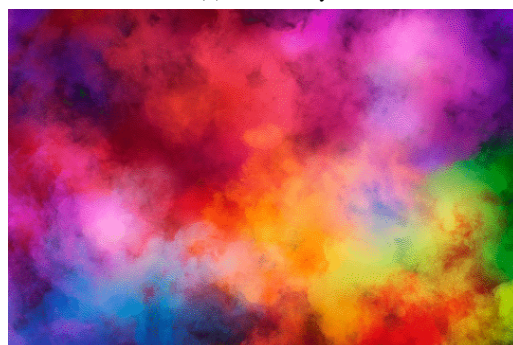
(a) Obraz RGB



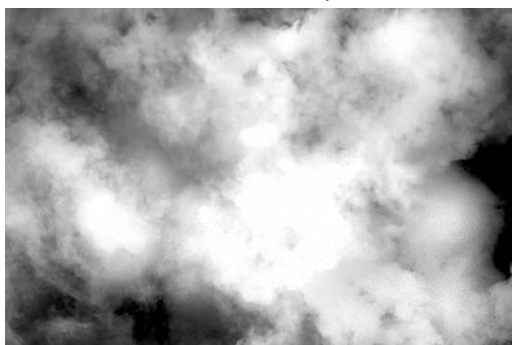
(b) Obraz szary



(c) Obraz binarny



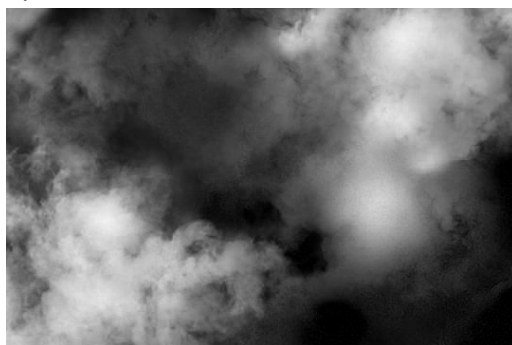
(d) Obraz indeksowany



(e) Kanał czerwony



(f) Kanał zielony



(g) Kanał niebieski

Rys. 1.1. Wizualizacja obrazów w różnych formatach i kanałach.

1.4.2. Zadanie 2

1.4.2.1. Treść

Zadanie 2. Histogram i operacje na histogramie.

W Matlab zaproponuj program, który pozwoli na:

- Wczytanie obrazu w skali szarości (imread → rgb2gray jeśli potrzeba).
- Obliczenie i wyświetlenie histogramu obrazu (imhist).
- Wykonaj rozciąganie histogramu (imadjust + stretchlim) i porównaj obraz przed i po operacji.
- Wykonaj wyrównanie histogramu (histeq) i porównaj wynik z oryginalnym obrazem oraz z histogramem rozciągniętym.
- Wyświetl histogramy wszystkich przekształconych obrazów.

1.4.2.2. Przykładowy program

Listing 1.2. Wczytanie obrazu i przetwarzanie obrazu histogramami

```
1 % Wczytanie obrazu (kolorowego lub w skali szarości)
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage); % Konwersja do skali szarości
4
5 % Wyświetlenie oryginalnego obrazu i jego histogramu
6 figure;
7 subplot(2,2,1), imshow(grayImage), title('Oryginalny obraz (gray)');
8 subplot(2,2,2), imhist(grayImage), title('Histogram oryginalny');
9
10 % Rozciąganie histogramu (stretching)
11 stretchedImage = imadjust(grayImage, stretchlim(grayImage));
12 subplot(2,2,3), imshow(stretchedImage), title('Po rozciąganiu histogramu');
13 subplot(2,2,4), imhist(stretchedImage), title('Histogram po rozciąganiu');
14
15 % Wyrównanie histogramu (equalization)
16 equalizedImage = histeq(grayImage);
17
18 % Nowa figura do porównań
19 figure;
20 subplot(2,2,1), imshow(equalizedImage), title('Po wyrównaniu histogramu');
21 subplot(2,2,2), imhist(equalizedImage), title('Histogram po wyrównaniu');
22 subplot(2,2,3), imshowpair(stretchedImage, equalizedImage, 'montage');
23 title('Porównanie: Rozciągnięty vs Wyrównany');
24
25 % Zapis obrazów (opcjonalnie)
26 imwrite(grayImage, 'zadanie2images/obraz_szary.png');
27 imwrite(stretchedImage, 'zadanie2images/obraz_stretched.png');
28 imwrite(equalizedImage, 'zadanie2images/obraz_equalized.png');
```

1.4.2.3. Obrazy



(a) Obraz wyrównany



(b) Obraz rozciągnięty

Rys. 1.2. Wizualizacja rozciągania i wyrównywania obrazów

1.4.2.4. Pytania do analizy

- Jaka jest różnica między rozciąganiem a wyrównywaniem histogramu?
 - Rozciąganie (stretching): zmienia zakres jasności, przekształcając piksele tak, by rozciągnąć istniejący histogram do pełnego zakresu (np. 0–255).
 - Wyrównywanie (equalization): zmienia rozkład jasności, aby histogram był możliwie równomierny – poprawiając kontrast globalnie.
- Która metoda daje lepsze wyniki poprawy kontrastu?
 - Dla obrazów o niskim kontraście — wyrównanie często działa lepiej.
 - Dla obrazów z dobrze rozłożonymi poziomami jasności — rozciąganie może być wystarczające i mniej inwazyjne.
- Jakie są potencjalne zastosowania tych technik?
 - Poprawa jakości obrazów w:
 - * medycynie (np. obrazowanie rentgenowskie),
 - * nadzorze wizyjnym (monitoring),
 - * przetwarzaniu zdjęć satelitarnych,
 - * OCR (rozpoznawanie tekstu),
 - * automatycznej analizie obrazów przemysłowych.

1.4.3. Zadanie 3

1.4.3.1. Treść

Zadanie 3. Segmentacja obrazu na podstawie histogramu.

W Matlab zaproponuj program, który pozwoli na:

- Wczytanie obrazu w skali szarości.
- Obliczenie histogramu obrazu (imhist) i znalezienie potencjalnego progu segmentacji.
- Przeprowadź ręczną segmentację obrazu, stosując różne wartości progowe ($BW = I > \text{threshold}$).
- Zastosuj automatyczną segmentację metodą Otsu (graythresh + imbinarize).
- Porównaj efekty różnych progowań, wyświetlając wyniki obok siebie.

1.4.3.2. Przykładowy program

Listing 1.3. Wczytanie obrazu i segmentacja na podstawie histogramu

```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage);
4
5 % Wyświetlenie histogramu
6 figure;
7 imhist(grayImage);
8 title('Histogram obrazu w skali szarości');
9
10 % Segmentacja ręczna przy różnych progach
11 threshold1 = 80;
12 threshold2 = 120;
13 threshold3 = 160;
14
15 BW1 = grayImage > threshold1;
16 BW2 = grayImage > threshold2;
17 BW3 = grayImage > threshold3;
18
19 % Segmentacja automatyczna metodą Otsu
20 otsuThreshold = graythresh(grayImage); % zwraca wartość w zakresie [0, 1]
21 BW_otсу = imbinarize(grayImage, otsuThreshold);
22
23 % Wyświetlenie wyników
24 figure;
25 subplot(2,3,1), imshow(grayImage), title('Oryginalny obraz');
26 subplot(2,3,2), imshow(BW1), title(['Próg ręczny = ' num2str(threshold1)]);
27 subplot(2,3,3), imshow(BW2), title(['Próg ręczny = ' num2str(threshold2)]);
28 subplot(2,3,4), imshow(BW3), title(['Próg ręczny = ' num2str(threshold3)]);
29 subplot(2,3,5), imshow(BW_otсу), title(['Otsu (próg = ' num2str(round(otsuThreshold*255))
    ' ')]);
30
31 % Zapis wyników (opcjonalnie)
32 imwrite(BW1, 'zadanie3images/progowanie_80.png');
33 imwrite(BW2, 'zadanie3images/progowanie_120.png');
34 imwrite(BW3, 'zadanie3images/progowanie_160.png');
35 imwrite(BW_otсу, 'zadanie3images/segmentacja_otсу.png');
```

1.4.3.3. Obrazy



(a) Progowanie 80



(b) Progowanie 120



(c) Progowanie 160



(d) Segmentacja Otsu

Rys. 1.3. Operacje segmentowanie progowaniem oraz otsu.

1.4.3.4. Pytania do analizy

- Jak dobrać optymalny próg segmentacji dla różnych obrazów?
 - **Ręcznie:** na podstawie histogramu – szukamy wyraźnych "dolinek" (oddzielenie klas).
 - **Automatycznie:** np. metoda Otsu – optymalizuje podział na dwie klasy o minimalnej wariancji wewnątrzklasowej.
- Kiedy metoda Otsu jest skuteczniejsza niż segmentacja ręczna?
 - Gdy histogram jest **dwumodalny** (dwa główne zbiory pikseli – tło i obiekt).
 - Gdy chcemy **zautomatyzować** segmentację wielu obrazów bez ręcznego ustawiania progów.
 - Gdy nie mamy wiedzy o zawartości obrazu i chcemy uzyskać ogólnie dobrą separację.

- Jakie mogą być praktyczne zastosowania binaryzacji?
 - Detekcja i zliczanie obiektów (np. komórek w mikroskopii).
 - OCR (rozpoznawanie tekstu).
 - Analiza konturów i kształtów.
 - Przemysł: detekcja defektów, systemy wizyjne.
 - Monitoring: wykrywanie ruchu, segmentacja obiektów.

1.4.4. Zadanie 4

1.4.4.1. Treść

Zadania 4: Detekcja obiektów na obrazie binarnym.

W Matlab zaproponuj program, który pozwoli na:

- Wykonasz segmentację obrazu metodą Otsu i uzyskasz obraz binarny.
- Zidentyfikuj obiekty na obrazie binarnym za pomocą funkcji `bwlabel` lub `regionprops`.
- Wyznacz podstawowe właściwości wykrytych obiektów (np. powierzchnię, obwód).
- Narysuj prostokątne obramowania wokół wykrytych obiektów (`rectangle`).
- Zwizualizuj oryginalny obraz z nałożonymi obramowaniami.

1.4.4.2. Przykładowy program

Listing 1.4. Wczytanie obrazu i dokonanie detekcji obiektów na obrazie

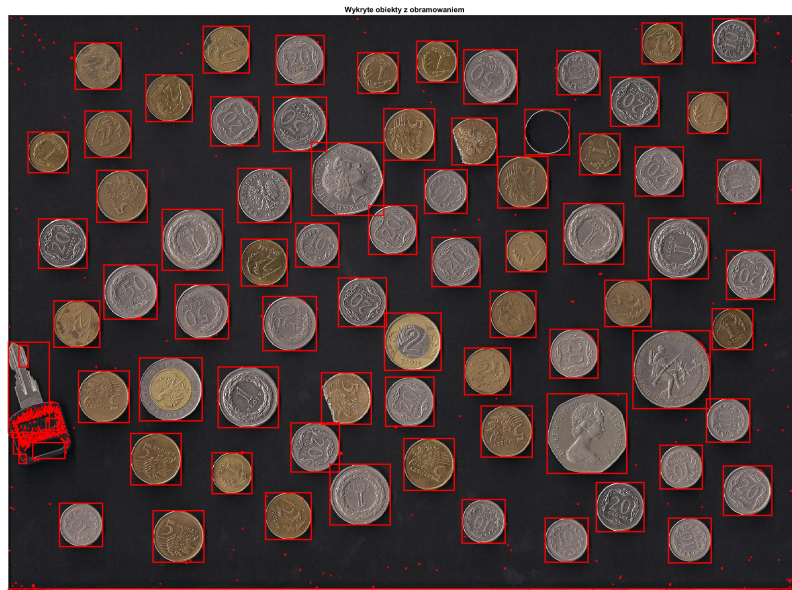
```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('monety1.jpg'); % przykład dobrze kontrastowego obrazu
3 grayImage = rgb2gray(rgbImage);
4
5 % Segmentacja metodą Otsu
6 binaryImage = imbinarize(grayImage, graythresh(grayImage));
7
8 % Wypełnienie dziur (opcjonalne)
9 binaryImage = imfill(binaryImage, 'holes');
10
11 % Etykietowanie obiektów
12 labeledImage = bwlabel(binaryImage);
13
14 % Właściwości obiektów
15 props = regionprops(labeledImage, 'BoundingBox', 'Area', 'Perimeter');
16
17 % Wyświetlenie obrazu z nałożonymi obramowaniami
18 figure;
19 imshow(rgbImage);
20 title('Wykryte obiekty z obramowaniem');
21 hold on;
22
23 % Iteracja przez obiekty i rysowanie obramowań
24 for k = 1:length(props)
25     bbox = props(k).BoundingBox;
26     rectangle('Position', bbox, 'EdgeColor', 'r', 'LineWidth', 2);
```

```

27
28 % Wyświetlanie powierzchni i obwodu w konsoli (opcjonalnie)
29 fprintf('Obiekt %d: Powierzchnia = %.2f, Obwód = %.2f\n', k, props(k).Area, props(k)
30 .Perimeter);
31
32 hold off;
33 exportgraphics(gca, 'zadanie4images/obiekty_z_obramowaniem.png');

```

1.4.4.3. Obrazy



Rys. 1.4. Detekcja obiektów na obrazie.

1.4.4.4. Pytania do analizy

1. Jakie problemy mogą wystąpić przy detekcji obiektów w zależności od jakości segmentacji?

- Zbyt niska jakość segmentacji może prowadzić do:
 - scalania sąsiadujących obiektów w jeden,
 - pocięcia jednego obiektu na kilka części,
 - pomijania obiektów o słabym kontraście,
 - szumu – detekcji fałszywych obiektów.

2. Jak można poprawić dokładność detekcji?

- Filtracja obrazu przed segmentacją (np. `medfilt2`, `imgaussfilt`).
- Morfologia: `imopen`, `imclose`, `bwareaopen` – usuwa drobne obiekty/szumy.
- Dostosowanie progu segmentacji ręcznie lub przez adaptacyjną binaryzację (`adaptthresh`).
- Użycie maski ROI lub filtrowanie po właściwościach (Area, Eccentricity, itp.).

3. Jakie są praktyczne zastosowania tej metody?

- **Biomedycyna:** zliczanie komórek, wykrywanie struktur w mikroskopii.
- **Przemysł:** kontrola jakości (np. wykrywanie wad, pomiar kształtu).

- **Robotyka/Widzenie maszynowe:** identyfikacja i lokalizacja obiektów.
- **Bezpieczeństwo:** wykrywanie tablic rejestracyjnych, ludzi, ruchu.

1.4.5. Zadanie 5

1.4.5.1. Treść

Zadanie 5. Segmentacja wieloprogowa

W Matlab zaproponuj program, który pozwoli na:

- Wczytasz obraz w skali szarości.
- Obliczysz histogram obrazu i znajdziesz dwa optymalne progi metodą Otsu (multithresh).
- Utwórz obraz segmentowany na trzy klasy (imquantize).
- Zwizualizuj wynik segmentacji z użyciem różnych kolorów (label2rgb).
- Porównaj segmentację wieloprogową z binaryzacją jednokryterialną.

1.4.5.2. Przykładowy program

Listing 1.5. Wczytanie obrazu i dokonanie detekcji obiektów na obrazie

```
1 % Wczytanie obrazu i konwersja do skali szarości
2 rgbImage = imread('cameraman.jpg');
3 grayImage = rgb2gray(rgbImage);
4
5 % Wyznaczenie 2 progów metodą Otsu (wieloprogowość)
6 thresholds = multithresh(grayImage, 2); % [t1, t2]
7
8 % Segmentacja obrazu na 3 klasy
9 segmentedImage = imquantize(grayImage, thresholds);
10
11 % Kolorowanie klas dla wizualizacji
12 rgbSegmented = label2rgb(segmentedImage, 'jet', 'k');
13
14 % Binarizacja klasyczna dla porównania
15 binaryImage = imbinarize(grayImage, graythresh(grayImage));
16
17 % Wyświetlenie wyników
18 figure;
19 subplot(2,2,1), imshow(grayImage), title('Oryginalny obraz (gray)');
20 subplot(2,2,2), imshow(binaryImage), title('Binarizacja jednokryterialna');
21 subplot(2,2,3), imshow(segmentedImage, []), title('Obraz segmentowany (3 klasy)');
22 subplot(2,2,4), imshow(rgbSegmented), title('Segmentacja wieloprogowa (kolor)');
23
24 % Zapis obrazów (opcjonalny)
25 imwrite(binaryImage, 'zadanie5images/binaryzacja_otsu.png');
26 imwrite(rgbSegmented, 'zadanie5images/segmentacja_wieloprogowa.png');
```

1.4.5.3. Obrazy



(a) Progowanie 80



(b) Progowanie 120

Rys. 1.5. Operacje segmentowanie wieloprogowego.

1.4.5.4. Pytania do analizy

1. W jakich przypadkach segmentacja wieloprogowa daje lepsze wyniki niż binaryzacja?
 - Gdy obraz zawiera **więcej niż dwa poziomy jasności** reprezentujące różne obiekty lub tło (np. tło, cień, obiekt).
 - Przy obrazach z **wieloma klasami intensywności** – np. zdjęcia medyczne, zdjęcia z mikroskopu, obrazy satelitarne.
2. Jakie są ograniczenia segmentacji wieloprogowej?
 - **Wymaga wiedzy** o liczbie klas (ile progów wyznaczyć).
 - **Czasochłonna** przy większej liczbie progów lub dużych obrazach.
 - Czasem nie sprawdza się przy słabym kontraście lub zaszumionym obrazie – klasy mogą się zlewać.
3. Jak można poprawić jakość segmentacji?
 - **Filtracja** przed segmentacją (imgaussfilt, medfilt2).
 - **Użycie morfologii** po segmentacji (imopen, imclose, bwareaopen).
 - Dobór progów ręcznie lub przez adaptacyjne metody (adaptthresh).
 - W połączeniu z analizą cech (regionprops) można odfiltrować obiekty niepasujące do oczekiwań.

1.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

- Histogram jest przydatnym narzędziem w analizie jakości obrazu biometrycznego.

- Wyrównanie histogramu poprawia kontrast i może zwiększać czytelność obrazu.
- Segmentacja oparta na histogramie, w szczególności metoda Otsu, pozwala na skuteczne wydzielenie istotnych cech obrazu.

2. Laboratorium 2

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	DETEKCJA KRAWĘDZI W OBRAZIE METODAMI SPLOTOWYMI.
Ćwiczenie nr:	02
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	26.04.2025
Data oddania ćwiczenia	13.05.2025

2.1. Cel ćwiczenia

Filtracja splotowa to jedno z podstawowych narzędzi przetwarzania obrazów, wykorzystywane do wygładzania, wykrywania krawędzi, redukcji szumów oraz innych operacji poprawiających jakość obrazu. Splot (ang. convolution) polega na przekształceniu obrazu poprzez nałożenie na niego maski (jądra) filtrującego. Celem laboratorium jest implementacja algorytmu splotowego w MATLAB-ie oraz zbadanie wpływu różnych filtrów na obraz.

2.2. Wstęp teoretyczny

2.2.0.1. Filtracja splotowa

Filtracja splotowa (ang. convolution filtering) jest podstawową metodą przetwarzania obrazów w dziedzinie przestrzennej. Polega ona na nakładaniu na obraz niewielkiej maski (kernela) i obliczaniu nowych wartości pikseli jako sumy ważonej sąsiednich pikseli zgodnie z wartościami w masce. W praktyce oznacza to, że dla każdego punktu obrazu uwzględnia się przyległe piksele, których wartości mnoży się przez odpowiadające wagi z maski, a następnie sumuje. Taki proces pozwala uzyskać różne efekty – od wygładzania (rozmycia) poprzez wyostanie po detekcję krawędzi, w zależności od przyjętych wag w masce filtra.

2.2.0.2. Filtry krawędziowe (Roberts, Prewitt, Sobel, Laplace)

Operator Roberta – jest jednym z pierwszych detektorów krawędzi wykorzystujących maski różnicowe. Używa dwóch małych masek 2×2 do przybliżenia gradientu obrazu w kierunkach ukośnych. Filtr ten jest bardzo prosty obliczeniowo (maski zawierają tylko liczby całkowite), jednak jego wadą jest duża wrażliwość na szum. Wynikiem działania operatora Roberta są zmiany intensywności wzdłuż przekątnych, co prowadzi do wykrywania krawędzi o orientacji diagonalnej.

Operator Prewitta – wykorzystuje dwie maski 3×3 do przybliżenia pochodnych obrazu w kierunku poziomym i pionowym. Każda z masek jest wynikiem splotu filtru różnicującego z filtrem uśredniającym. Dzięki temu operator Prewitta częściowo wygładza obraz, co poprawia wykrywanie krawędzi poziomych i pionowych. Generowany wektor gradientu obrazu wskazuje siłę krawędzi (długość wektora) oraz jej orientację (kierunek wektora).

Operator Sobela – podobnie jak Prewitt, używa masek 3×3 , jednak nadaje większe wagi środkowym elementom masek (wartość 2), co skutkuje silniejszym wygładzeniem i zwiększoną odpornością na szum. Operator Sobela wydajnie przybliża gradient intensywności obrazu, szczególnie eksponując miejsca gwałtownych zmian jasności.

Operator Laplace’a – oparty jest na drugiej pochodnej obrazu (Laplasjanie) i wykrywa zmiany intensywności niezależnie od kierunku. Jest to filtr drugiego rzędu, izotropowy w przestrzeni 2D. Operator Laplace’a uwypukla obszary szybkiej zmiany jasności, ale jest również podatny na szumy. Charakterystyczną cechą wyników Laplasjanu jest wykrywanie miejsc przecięcia zera (zero-crossings), które odpowiadają lokalizacjom krawędzi.

2.2.0.3. Filtr medianowy a filtr uśredniający

Filtr medianowy jest nieliniowym filtrem, w którym wartość piksela zastępowana jest medianą wartości z sąsiadujących pikseli. Dzięki temu skutecznie usuwa szum impulsowy typu „sól i pieprz”, zachowując jednocześnie ostrość krawędzi. W przeciwieństwie do filtrów liniowych, filtr medianowy nie wprowadza nowych wartości spoza przedziału istniejących intensywności, co zapobiega powstawaniu artefaktów.

Filtr uśredniający (średnia arytmetyczna) jest filtrem liniowym, który zastępuje wartość piksela średnią z jego sąsiadów. Filtr ten skutecznie redukuje szumy o rozkładzie Gaussa, jednak prowadzi do rozmycia krawędzi i utraty szczegółów. W praktyce filtr medianowy jest preferowany do usuwania szumu impulsowego, natomiast filtr uśredniający stosuje się głównie do ogólnego wygładzania obrazu.

2.2.0.4. Szum typu „sól i pieprz” (impulsowy)

Szum typu „sól i pieprz” to rodzaj szumu impulsowego, w którym losowe piksele przyjmują wartości minimalne (0, „pieprz”) lub maksymalne (255, „sól”). Zazwyczaj występuje na skutek błędów transmisji, uszkodzeń matrycy detektora lub błędów zapisu danych. Do jego usunięcia najlepiej nadają się filtry medianowe, które skutecznie eliminują pojedyncze anomalie bez znaczącej utraty ostrości obrazu. Filtry liniowe, takie jak filtr uśredniający, są mniej skuteczne, ponieważ rozmywają szum na większe obszary obrazu.

2.3. Opis wykorzystywanego oprogramowania i narzędzi

Do przeprowadzenia eksperymentu wykorzystano środowisko MATLAB oraz bibliotekę Image Processing Toolbox. Kluczowe funkcje używane w analizie to:

- `imfilter()` – funkcja MATLAB służąca do filtracji obrazów za pomocą splotu lub korelacji z podaną maską. Pozwala na wybór sposobu obramowania granic obrazu oraz zachowuje typ danych wejściowych. Funkcja `imfilter()` umożliwia wygodne operacje na obrazach, zachowując ich oryginalne rozmiary.
- `conv2()` – funkcja realizująca dwuwymiarowy klasyczny splot, domyślnie zwracająca wynik o większym rozmiarze niż obraz wejściowy. Wynikowy obraz jest typu zmiennoprzecinkowego (double). `conv2()` jest wydajniejsza dla dużych masek, jednak wymaga ręcznego dostosowania rozmiaru i rzutowania typu wynikowego.

2.4. Zadania do wykonania samodzielnego

2.4.1. Zadanie 1

2.4.1.1. Treść

Filtrowanie splotowe obrazu oka:

Wykorzystując dowolny obraz biometryczny tęczówki oka wykonaj program dokonujący filtracji splotowej filtrami Robertsa, Prewitta, Sobela oraz Laplace'a. Uwzględnij w programie możliwość zadawania rozmiaru maski kernela 3x3 oraz 5x5.

- Wyświetl wyniki filtracji w oknie wykorzystując funkcję subplot.
- Porównaj efekty filtracji dla różnych filtrów. Sprawdź wpływ zmiany rozmiaru maski filtra na wynik końcowy.
- Wprowadź do obrazu zadaną porcję szumu SaltAndPepper wykorzystując funkcję imnoise.
- Przeprowadź na obrazie wejściowym filtrację medianową i uśredniającą a następnie porównaj otrzymane wyniki. Ponów procedurę wykrycia krawędzi w obrazie; przedstaw w sprawozdaniu dyskusję otrzymanych wyników.
- Wczytaj obraz kolorowy (peppers.png) i zastosuj filtrację na każdej składowej R, G, B osobno. Połącz prefiltrowane składowe w nowy obraz i wyświetl wynik.
- Sprawdź, jak działa imfilter(I, h, 'same') w porównaniu do conv2(). Czy wyniki są identyczne? Jakie są różnice?

2.4.1.2. Przykładowy program

Listing 2.1. Wczytanie obrazu i stworzenie konwersji

```
1 % Tworzenie katalogu, jeśli nie istnieje
2 outputFolder = 'zadanielimages';
3 if ~exist(outputFolder, 'dir')
4     mkdir(outputFolder);
5 end
6
7 % Wczytaj obraz biometryczny
8 I = imread('oko1.png');
9 I_gray = im2gray(I);
10 I_gray = im2double(I_gray);
11
12 % Definicje masek
13 roberts_mask = [0 0 0; -1 0 0; 0 1 0];
14 prewitt_mask = [-1 -1 -1; 0 0 0; 1 1 1];
15 sobel_mask = [-1 -2 -1; 0 0 0; 1 2 1];
16 laplace_mask1 = [0 -1 0; -1 4 -1; 0 -1 0];
17 laplace_mask2 = [-1 -2 -1; -2 4 -2; -1 -2 -1];
18
19 %% Filtracja 3x3
20 figure;
21 subplot(2,3,1), imshow(I_gray), title('Oryginał');
22 subplot(2,3,2), imshow(imfilter(I_gray, roberts_mask, 'replicate')), title('Roberts');
```

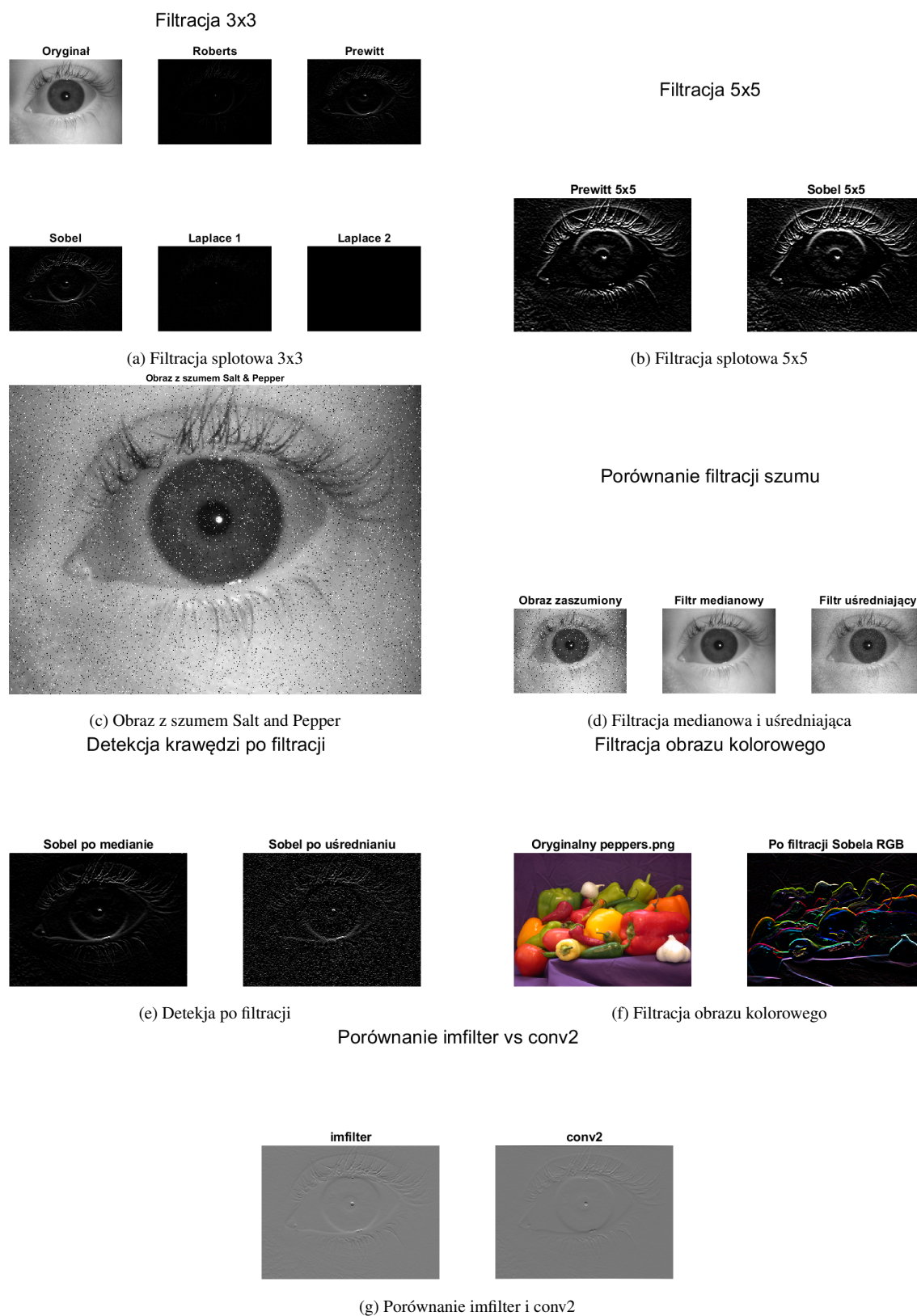
```

23 subplot(2,3,3), imshow(imfilter(I_gray, prewitt_mask, 'replicate')), title('Prewitt');
24 subplot(2,3,4), imshow(imfilter(I_gray, sobel_mask, 'replicate')), title('Sobel');
25 subplot(2,3,5), imshow(imfilter(I_gray, laplace_mask1, 'replicate')), title('Laplace 1')
    ;
26 subplot(2,3,6), imshow(imfilter(I_gray, laplace_mask2, 'replicate')), title('Laplace 2')
    ;
27 sgtitle('Filtracja 3x3');
28
29 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_3x3.png'));
30
31 %% Filtracja 5x5
32 hprewitt5 = imresize(fspecial('prewitt'), [5 5], 'nearest');
33 hsobel5 = imresize(fspecial('sobel'), [5 5], 'nearest');
34
35 figure;
36 subplot(1,2,1), imshow(imfilter(I_gray, hprewitt5, 'replicate')), title('Prewitt 5x5');
37 subplot(1,2,2), imshow(imfilter(I_gray, hsobel5, 'replicate')), title('Sobel 5x5');
38 sgtitle('Filtracja 5x5');
39
40 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_5x5.png'));
41
42 %% Dodanie szumu Salt & Pepper
43 noisy = imnoise(I_gray, 'salt & pepper', 0.05);
44
45 figure;
46 imshow(noisy);
47 title('Obraz z szumem Salt & Pepper');
48
49 exportgraphics(gcf, fullfile(outputFolder, 'szum_salt_pepper.png'));
50
51 %% Filtracja medianowa i uśredniająca
52 median_filtered = medfilt2(noisy, [3 3]);
53 average_filtered = imfilter(noisy, fspecial('average', [3 3]), 'replicate');
54
55 figure;
56 subplot(1,3,1), imshow(noisy), title('Obraz zaszumiony');
57 subplot(1,3,2), imshow(median_filtered), title('Filtr medianowy');
58 subplot(1,3,3), imshow(average_filtered), title('Filtr uśredniający');
59 sgtitle('Porównanie filtracji szumu');
60
61 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_szumu.png'));
62
63 %% Detekcja krawędzi po filtracji
64 figure;
65 subplot(1,2,1), imshow(imfilter(median_filtered, sobel_mask, 'replicate')), title('Sobel
    po medianie');
66 subplot(1,2,2), imshow(imfilter(average_filtered, sobel_mask, 'replicate')), title('
    Sobel po uśrednianiu');
67 sgtitle('Detekcja krawędzi po filtracji');
68
69 exportgraphics(gcf, fullfile(outputFolder, 'detekcja_krawedzi_po_filtracji.png'));
70
71 %% Filtracja obrazu kolorowego peppers.png
72 RGB = imread('peppers.png');
73 R = RGB(:, :, 1);
74 G = RGB(:, :, 2);
75 B = RGB(:, :, 3);

```

```
76
77 Rf = imfilter(R, sobel_mask, 'replicate');
78 Gf = imfilter(G, sobel_mask, 'replicate');
79 Bf = imfilter(B, sobel_mask, 'replicate');
80
81 filtered_RGB = cat(3, Rf, Gf, Bf);
82
83 figure;
84 subplot(1,2,1), imshow(IMG), title('Oryginalny peppers.png');
85 subplot(1,2,2), imshow(filtered_RGB, []), title('Po filtracji Sobela RGB');
86 sgtitle('Filtracja obrazu kolorowego');
87
88 exportgraphics(gcf, fullfile(outputFolder, 'filtracja_rgb.png'));
89
90 %% Porównanie imfilter vs conv2
91 imf = imfilter(I_gray, sobel_mask, 'same', 'replicate');
92 convf = conv2(I_gray, sobel_mask, 'same');
93
94 figure;
95 subplot(1,2,1), imshow(imf, []), title('imfilter');
96 subplot(1,2,2), imshow(convf, []), title('conv2');
97 sgtitle('Porównanie imfilter vs conv2');
98
99 exportgraphics(gcf, fullfile(outputFolder, 'porownanie_imfilter_conv2.png'));
```

2.4.1.3. Obrazy



Rys. 2.1. Wizualizacja obrazów w różnych filtracjach splotowych.

2.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

- **Filtry krawędziowe (Roberts, Prewitt, Sobel, Laplace)** skutecznie wykrywają zmiany intensywności w obrazie, jednak ich charakterystyka znacząco się różni. Operator Roberta jest prosty i szybki obliczeniowo, lecz wykazuje dużą wrażliwość na szum i słabo radzi sobie z krawędziami innymi niż ukośne. Operator Prewitta oraz Sobela są bardziej odpornymi filtrami, przy czym filtr Sobela oferuje lepsze wygładzenie dzięki większemu obciążeniu środkowych pikseli. Operator Laplace’a umożliwia wykrycie krawędzi niezależnie od kierunku, ale jego wrażliwość na szumy jest jeszcze większa niż filtrów pierwszorzędowych.
- **Rozmiar maski filtra** wpływa na wynik filtracji: maski 5x5 zapewniają mocniejsze wygładzanie i bardziej rozmyte krawędzie w porównaniu do masek 3x3. Zwiększenie rozmiaru maski powoduje, że filtr staje się mniej czuły na drobne detale i szum, ale za to mniej precyzyjnie lokalizuje krawędzie.
- **Wpływ szumu Salt&Pepper** na jakość obrazu jest znaczny. Obecność tego typu zakłóceń prowadzi do licznych, losowych zaburzeń w obrazie, które mogą zostać błędnie zinterpretowane jako krawędzie podczas procesu filtracji.
- **Filtr medianowy** wykazuje wysoką skuteczność w usuwaniu szumu typu Salt&Pepper przy zachowaniu krawędzi oraz struktury obrazu. W przeciwieństwie do niego, **filtr uśredniający** rozmywa obraz, prowadząc do rozmycia granic i mniejszej skuteczności w eliminacji impulsowych zakłóceń.
- **Filtracja obrazu kolorowego** (na osobnych kanałach RGB) umożliwia skuteczne przetwarzanie obrazów wielokanałowych. Jednak filtracja niezależnie każdego kanału może prowadzić do powstania artefaktów kolorystycznych, zwłaszcza w obszarach o silnych zmianach koloru.
- **Porównanie funkcji `imfilter` oraz `conv2`** wykazało, że obie metody dają podobne wyniki przy odpowiednim doborze opcji (parametr `'same'`). Funkcja `imfilter` jest bardziej elastyczna, oferując możliwość wyboru metody obsługi brzegów obrazu oraz zachowania typu danych. Z kolei `conv2` jest bardziej „surowa” i wymaga ręcznego dostosowania typu i rozmiaru wyniku.

Podsumowując, odpowiedni dobór metody filtracji, rozmiaru maski oraz sposobu redukcji szumów ma kluczowe znaczenie dla skutecznej analizy obrazów biometrycznych i przetwarzania obrazów cyfrowych w ogóle.

3. Laboratorium 3

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	OPERACJE MORFOLOGICZNE NA OBRAZACH
Ćwiczenie nr:	03
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	26.04.2025
Data oddania ćwiczenia	13.05.2025

3.1. Cel ćwiczenia

Operacje morfologiczne są kluczowym narzędziem w przetwarzaniu obrazów, szczególnie w analizie kształtów i struktury obiektów. Dzięki wykorzystaniu odpowiednich elementów strukturalnych można dostosować te operacje do różnych zadań, takich jak segmentacja, poprawa jakości obrazu czy analiza konturów. Celem laboratorium jest zapoznanie się z operacjami morfologicznymi na obrazach.

3.2. Wstęp teoretyczny

Operacje morfologiczne to techniki przetwarzania obrazów stosowane głównie do analizy kształtu i struktury obiektów w obrazach binarnych oraz, w pewnym stopniu, w obrazach w skali szarości. Ich główną cechą jest wykorzystanie elementów strukturalnych do modyfikowania obrazu.

Szkieletyzacja to proces odnoszący się do ścieniania linii konturowych obiektu, który znalazł zastosowanie w wielu dziedzinach, takich jak:

- medycyna (analiza białych ciałek krwi, chromosomów, naczyń wieńcowych),
- kryminalistyka (rozpoznawanie odcisków palców),
- przemysł elektroniczny (analiza obwodów drukowanych),
- wektoryzacja obrazów rastrowych.

W niniejszym opracowaniu przedstawiono sposób działania wybranych algorytmów szkieletyzujących: Arcelli-Levialdi-Pavlidis, Chin-Van-Stover-Iverson oraz Rosenfeld-Peleg. Zaprojektowano także aplikację umożliwiającą użytkownikowi wprowadzanie własnych szablonów szkieletyzujących oraz obserwowanie otrzymanych wyników.

3.2.1. Szkieletyzacja obrazów binarnych

Pośród klasycznych metod ścieniania (*skeletoning* lub *thinning algorithms*), wyróżnić można trzy główne podejścia:

- **Wypełnianie działów wodnych** (dla obrazów w skali szarości),

- **Hit or Miss Transform** (dla obrazów binarnych i szarych),
- **Medial Axis Transform** (posiadająca przekształcenie odwrotne).

Podstawą tych metod są dwa główne przekształcenia morfologiczne: erozja i dylatacja. Formalnie, dla obrazów binarnych:

$$A \oplus B = \bigcup_{b \in B} \left(\bigcup_{a \in A} (a + b) \right) \quad (3.1)$$

$$A \ominus B = \bigcap_{b \in B} \left(\bigcup_{a \in A} (a + b) \right) \quad (3.2)$$

gdzie A to obraz, B to maska (element strukturalny), $\oplus$ oznacza dylatację, a $\ominus$ erozję. Znak $\bigcup$ oznacza sumę zbiorów, a $\bigcap$ ich część wspólną.

Hit or Miss Transform można opisać następująco:

- przykładamy element strukturalny b do każdego punktu obrazu,
- jeśli b pokrywa się z sąsiedztwem - zmieniamy wartość piksela (zwykle na 0),
- w przeciwnym wypadku piksel pozostaje niezmienny.

Algorytmy takie jak Arcelli-Levialdi stosują szablony operujące na oknie bieżącego piksela oraz jego ośmiu sąsiadów. Możliwa jest także modyfikacja szablonów (np. użycie bitów typu "don't care") w celu uzyskania lepszego efektu szkieletyzacji gałęzi pionowych.

3.2.2. Algorytm Chin-Van-Stover-Iverson

Algorytm CVSI porównuje sąsiedztwo piksela z zestawem szablonów (1–8). Jeśli spełniony jest warunek zgodności z szablonem, następuje dodatkowe porównanie z szablonami 9 i 10.

- Jeśli otoczenie nie różni się od żadnego z szablonów — piksel pozostaje (jest częścią szkieletu).
- Jeśli różni się — piksel zostaje usunięty.

Aplikacja umożliwia ręczną edycję szablonów przez użytkownika (zmiana wartości 0, 1, x za pomocą kliknięcia).

3.2.3. Szkieletyzacja obrazów tonowanych

Naturalnym rozwinięciem szkieletyzacji obrazów binarnych jest szkieletyzacja obrazów w skali szarości. Przykładowe podejścia:

- erozja (Rosenfeld-Peleg),
- przekształcenie typu top-hat.

Ogólna translacja sygnału $f(x)$:

$$f(x) = f(x - z) \quad (3.3)$$

$$(f \oplus y)(x) = f(x) + y \quad (3.4)$$

Obie translacje można łączyć:

$$(f_z \oplus y)(x) = f(x - z) + y \quad (3.5)$$

Jeśli przyjmujemy, że $f(x) = -\infty$ poza obrazem, to dziedzinę obrazu określimy jako:

$$D[f] = \{x : f(x) > -\infty\} \quad (3.6)$$

Jeżeli $g(x) \leq f(x)$ dla każdego x , to:

$$D[g] \subseteq D[f] \quad (3.7)$$

Erozję definiujemy jako:

$$(f \wedge g)(x) = \begin{cases} \min\{f(x), g(x)\}, & x \in D[f] \cap D[g] \\ -\infty, & \text{inaczej} \end{cases} \quad (3.8)$$

Przy zastosowaniu płaskiego elementu strukturyzującego ($g_z(x) = 0$), erozję można zapisać jako:

$$(f \ominus g)(x) = \min\{f(x) - g_z(x)\} \quad (3.9)$$

co odpowiada filtracji typu *moving minimum*.

Do obserwacji zachowania algorytmu Rosenfeld-Peleg zaprojektowano obraz syntetyczny zawierający cylindryczną falę sinusoidalną.

3.3. Zadania do wykonania samodzielnego

3.3.1. Zadanie 1

3.3.1.1. Treść

1. Załaduj obraz testowy oraz przygotuj obraz do realizacji dalszych ćwiczeń o rozmiarach:
2. Wykonaj program realizujący algorytm ścieniania, którego zestaw kerneli zamieszczono poniżej:
3. Do zaprogramowania kolejnych szablonów i masek wykorzystaj poniższy fragment kodu źródłowego (funkcja `intbin8` formuje 8-bitową reprezentację liczby dziesiętnej przekazywanej parametrem `i`):
4. Sprawdź poprawność działania algorytmu na innym obrazie testowym, dla zmodyfikowanych parametrów filtracji morfologicznej jak poniżej:
5. Wyświetl wyniki otrzymanych filtracji wraz z podpisem w oknie wykorzystując funkcję `subplot`.
6. Wyjaśnij zjawisko zmieniającej się szybkości wykonywania algorytmu w zależności od fazy jego wykonywania.
7. Wykonaj wersję algorytmu realizującego szkieletyzację wg Chin-Wan-Stover-Iverson, przyjmując zestaw kerneli jak poniżej
8. Przeprowadź dyskusję otrzymanych wyników jak dla pkt 5-6.

3.3.1.2. Przykładowy program

Listing 3.1. Wczytanie obrazu i stworzenie konwersji

```

1 %% Zadanie - Ścienianie + Szkieletyzacja Chin-Wan-Stover-Iverson
2
3 clear; close all; clc;
4
5 %% 1. Wczytanie obrazu
6 [I, map] = imread('finger.jpg');
7 I = im2bw(I); % jeśli obraz nie jest jeszcze binarny
8
9 % Funkcja pomocnicza - konwersja liczby na 8-bitowy wektor binarny
10 intbin8 = @(i) bitget(uint8(i), 8:-1:1);
11
12 %% 2. Definicja masek (standardowe ścienianie)
13 s = [192 80 12 5 3 65 48 20];
14 m = [206 87 236 117 59 93 179 213];
15
16 %% 3. Funkcja thinning (standardowa)
17 function J = thinning(I, s, m)
18     J = I;
19     done = false;
20     while ~done
21         marker = false(size(J));
22         for k = 1:length(s)
23             % Przygotuj sąsiedztwo 8-nearest
24             N = circshift(J, [-1, 0]);
25             NE = circshift(J, [-1, 1]);
26             E = circshift(J, [0, 1]);
27             SE = circshift(J, [1, 1]);
28             S_ = circshift(J, [1, 0]);
29             SW = circshift(J, [1, -1]);
30             W = circshift(J, [0, -1]);
31             NW = circshift(J, [-1, -1]);
32
33             neighbors = cat(3, N, NE, E, SE, S_, SW, W, NW);
34
35             % Konwersja na 8-bitowe liczby
36             binImage = zeros(size(J), 'uint8');
37             for bit = 1:8
38                 binImage = bitor(binImage, uint8(neighbors(:, :, bit)) * 2^(8-bit));
39             end
40
41             % Warunek maski
42             cond = bitand(binImage, uint8(m(k))) == uint8(s(k));
43             marker = marker | (cond & J);
44         end
45         J_old = J;
46         J(marker) = 0;
47         done = isequal(J, J_old);
48     end
49 end
50
51 %% 4. Ścienianie - standardowe i zmodyfikowane parametry
52 J_standard = thinning(I, s, m);
53

```

```

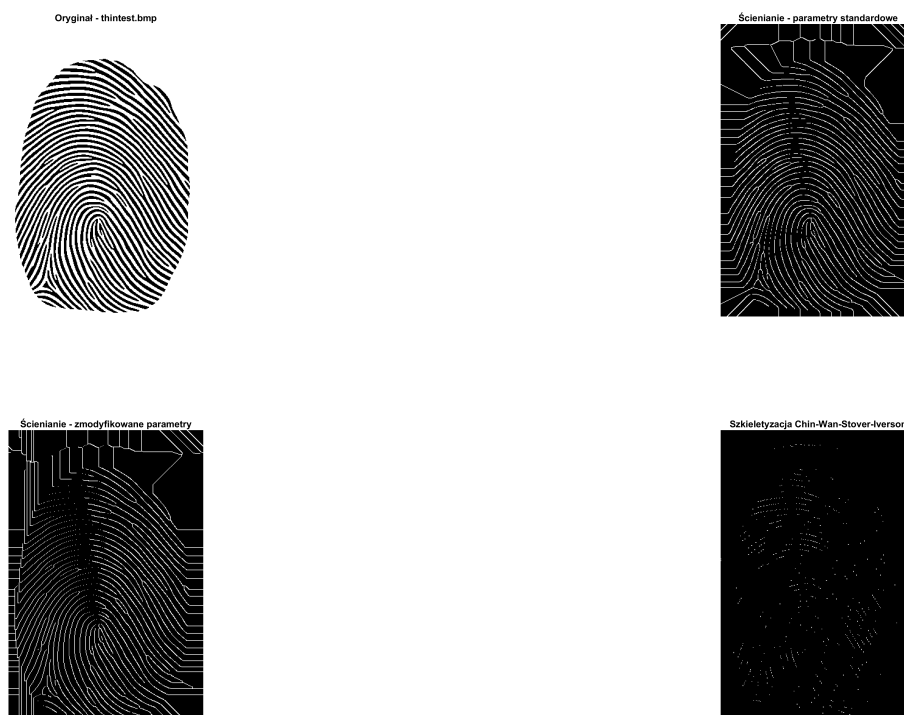
54 % Zmodyfikowane parametry
55 s_mod = [192 2 12 5 3 65 48 20];
56 m_mod = [206 58 236 117 59 93 179 213];
57 J_modified = thinning(I, s_mod, m_mod);
58
59 %% 5. Szkieletyzacja Chin-Wan-Stover-Iverson
60
61 % (przekształcenie na binarne maski)
62 s_cwsi = [0 1 1 0 0 1 0 0 1 1];
63 m_cwsi = [0 1 1 0 0 1 0 0 1 1];
64
65 % Funkcja szkieletyzacji
66 function J = cws_i_skeletonization(I)
67     J = I;
68     done = false;
69     while ~done
70         marker = false(size(J));
71
72         % Sąsiedztwo 8
73         N = circshift(J, [-1, 0]);
74         NE = circshift(J, [-1, 1]);
75         E = circshift(J, [0, 1]);
76         SE = circshift(J, [1, 1]);
77         S_ = circshift(J, [1, 0]);
78         SW = circshift(J, [1, -1]);
79         W = circshift(J, [0, -1]);
80         NW = circshift(J, [-1, -1]);
81
82         % Formowanie wektora sąsiadów
83         neighbors = [N(:) NE(:) E(:) SE(:) S_( :) SW(:) W(:) NW(:)];
84
85         % Liczenie warunków
86         % Przykład uproszczony - możesz zaimplementować bardziej dokładne odwzorowanie
            według tabelki
87         sum_neighbors = sum(neighbors, 2);
88         marker_vec = (sum_neighbors >= 2) & (sum_neighbors <= 6);
89         marker = reshape(marker_vec, size(J));
90
91         % Aktualizacja
92         J_old = J;
93         J(marker & J) = 0;
94         done = isequal(J, J_old);
95     end
96 end
97
98 J_cwsi = cws_i_skeletonization(I);
99
100 %% 6. Wyświetlenie wyników
101 figure('Name', 'Porównanie wyników', 'Units', 'normalized', 'OuterPosition', [0 0 1 1]);
102
103 subplot(2,2,1);
104 imshow(I); title('Oryginał - thintest.bmp');
105
106 subplot(2,2,2);
107 imshow(J_standard); title('Ścienianie - parametry standardowe');
108
109 subplot(2,2,3);

```

```

110 imshow(J_modified); title('Ścienianie - zmodyfikowane parametry');
111
112 subplot(2,2,4);
113 imshow(J_cwsi); title('Szkieletyzacja Chin-Wan-Stover-Iverson');
114
115 % --- Save figure with high quality ---
116 exportgraphics(gcf, 'results.png', 'Resolution', 300);
117
118 %% 7. Dyskusja wyników
119 %
120 % W algorytmie standardowego ścieniania obraz zostaje redukowany warstwa po warstwie,
121 % a przy zmodyfikowanych parametrach proces jest bardziej agresywny.
122 %
123 % W przypadku szkieletyzacji Chin-Wan-Stover-Iverson widać szybsze uproszczenie struktur
124 % ,
125 % co skutkuje cienkimi, pojedynczymi liniami, bez fragmentacji dużych obszarów.
126 %
127 % Zmienność szybkości działania algorytmu wynika z faktu, że w początkowych iteracjach
128 % usuwanych jest więcej pikseli, a w końcowych tylko pojedyncze piksele - co powoduje
    
```

3.3.1.3. Obrazy



Rys. 3.1. Porównanie wyników: Oryginał, Ścienianie standardowe, Ścienianie zmodyfikowane, Szkieletyzacja CWSI.

3.4. Wnioski

W przeprowadzonym ćwiczeniu dokonano analizy działania dwóch metod ścieniania obrazów binarnych:

- Metody klasycznej ścieniania z wykorzystaniem standardowych oraz zmodyfikowanych masek.

- Metody szkieletyzacji Chin-Wan-Stover-Iverson.

Na podstawie uzyskanych wyników można wyciągnąć następujące wnioski:

1. **Ścienianie z maskami standardowymi** pozwala na sukcesywne usuwanie zbędnych pikseli z obrazu, prowadząc do uzyskania cieńszych struktur. Proces ten jest stosunkowo stabilny, zachowując kształt linii papilarnych.
2. **Zmodyfikowane parametry ścieniania** powodują bardziej agresywne usuwanie pikseli. Widoczne jest zwiększone uproszczenie obrazu, jednak miejscami może to prowadzić do zniekształcenia struktury oryginalnych linii.
3. **Szkieletyzacja metodą Chin-Wan-Stover-Iverson** skutkuje szybkim i skutecznym redukowaniem obrazu do jego podstawowych konturów. Uzyskany szkielet jest bardzo cienki, jednak często prowadzi do fragmentacji obrazu, zwłaszcza w miejscach o większej gęstości pikseli.
4. **Szybkość działania algorytmu** jest największa na początku, gdy usuwane są większe grupy pikseli. W późniejszych iteracjach proces znacznie zwalnia, ponieważ zmiany obejmują pojedyncze piksele.
5. Obie metody pozwalają na skuteczne ścienianie obrazów binarnych, jednak wybór odpowiedniej metody oraz parametrów powinien być dostosowany do specyfiki przetwarzanego obrazu oraz wymagań aplikacji końcowej (np. analizy odcisków palców, rozpoznawania wzorców itp.).

Dzięki zastosowaniu różnych wariantów masek oraz metod możliwe jest dostosowanie procesu ścieniania do indywidualnych potrzeb, co stanowi istotny aspekt w cyfrowym przetwarzaniu obrazów.

4. Laboratorium 4

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	TRANSFORMATATA HOUGHGA
Ćwiczenie nr:	04
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	27.04.2025
Data oddania ćwiczenia	13.05.2025

4.1. Cel ćwiczenia

Celem zajęć jest zapoznanie się z transformacją Hough'a w zadaniu wykrywania linii i okręgów w testowych obrazach binarnych oraz obrazach tonowanych pozyskiwanych ze skanerów tęczówki oka.

4.2. Wstęp teoretyczny

4.2.1. Transformata Hougha

Transformata Hougha to technika analizy obrazu wykorzystywana do wykrywania prostych, okręgów oraz innych kształtów w obrazach cyfrowych. Polega na przekształceniu punktów z przestrzeni obrazu do tzw. przestrzeni parametrów, gdzie wykrywane są lokalne maksima odpowiadające obecności określonych kształtów.

4.2.1.1. Transformata Hougha – wersja liniowa

W przypadku wykrywania prostych, transformata Hougha przekształca punkt (x, y) z przestrzeni kartezjańskiej na krzywą w przestrzeni parametrów (ρ, θ) . Wykorzystuje się do tego normalną postać równania prostej:

$$\rho = x \cdot \cos(\theta) + y \cdot \sin(\theta), \quad (4.1)$$

gdzie:

- ρ – odległość prostej od początku układu współrzędnych,
- θ – kąt między osią x a normalną do prostej.

Każdy punkt z obrazu generuje w przestrzeni parametrów sinusoidę. Jeżeli kilka punktów leży na tej samej prostej, ich odpowiadające sinusoidy przecinają się w jednym punkcie – odpowiadającym parametrom tej prostej.

Aby odwzorować wszystkie możliwe proste, tworzy się macierz akumulatorową, w której zliczane są głosy (inkrementacje) za danymi parametrami (ρ, θ) . Miejsca o największej liczbie głosów odpowiadają wykrytym prostym.

Alternatywnie można stosować prostszą, choć mniej stabilną numerycznie formę liniową:

$$y = ax + b, \quad (4.2)$$

którą przekształca się do:

$$b = -ax + y. \quad (4.3)$$

W przestrzeni (a, b) każdemu punktowi z obrazu odpowiada linia prosta. Przecięcia tych prostych w jednym punkcie oznaczają wspólne parametry linii przechodzącej przez te punkty.

Algorytm detekcji linii – etapy:

1. Przekształcenie obrazu do postaci binarnej.
2. Utworzenie macierzy akumulatorowej (ρ, θ) .
3. Iteracja po wszystkich punktach krawędziowych i zliczanie głosów.
4. Wyszukiwanie lokalnych maksimów w akumulatorze.
5. Odwzorowanie wykrytych linii na obraz.

4.2.1.2. Transformata Hougha – wersja kołowa

Transformata Hougha może być również używana do detekcji okręgów. Równanie okręgu na płaszczyźnie opisuje się wzorem:

$$(x - a)^2 + (y - b)^2 = r^2, \quad (4.4)$$

gdzie:

- (a, b) – współrzędne środka okręgu,
- r – promień okręgu.

W tym przypadku przestrzeń parametrów jest trójwymiarowa: (a, b, r) . Każdy punkt z krawędzi obrazu może należeć do wielu potencjalnych okręgów, dlatego głosuje się dla wszystkich możliwych kombinacji (a, b, r) spełniających równanie okręgu.

Obliczenie współrzędnych środka okręgu dla ustalonego promienia r można przeprowadzić według wzorów:

$$a = x - r \cdot \cos(\theta), \quad (4.5)$$

$$b = y - r \cdot \sin(\theta). \quad (4.6)$$

Algorytm detekcji okręgów – etapy:

1. Wstępne przetwarzanie obrazu: konwersja do skali szarości, filtr Canny’ego, filtr Gaussa (opcjonalnie).
2. Inicjalizacja trójwymiarowego akumulatora (a, b, r) .
3. Iteracja po punktach krawędzi i promieniach, zliczanie głosów.
4. Wyszukiwanie lokalnych maksimów w akumulatorze.
5. Rysowanie wykrytych okręgów na obrazie.

Złożoność i optymalizacja

Transformata Hougha dla okręgów jest bardziej złożona obliczeniowo niż wersja liniowa, ponieważ wymaga analizowania trójwymiarowej przestrzeni parametrów. W celu optymalizacji stosuje się:

- ograniczenie zakresu promieni $r \in [r_{\min}, r_{\max}]$,
- metody probabilistyczne (np. Randomized Hough Transform),
- wykorzystanie gradientu przyspieszającego głosowanie.

4.3. Opis wykorzystywanego oprogramowania i narzędzi

W realizacji zadania wykorzystano szereg funkcji dostępnych w środowisku MATLAB, które są szczególnie przydatne w analizie obrazów oraz implementacji transformaty Hougha:

- `hough(BW)` – wykonuje transformatę Hougha dla obrazu binarnego BW. Funkcja zwraca macierz akumulatora parametrów ρ i θ .
- `houghpeaks(H, N)` – znajduje lokalne maksima w macierzy akumulatora H. Argument N określa liczbę wykrywanych maksimów. Funkcja zwraca współrzędne punktów odpowiadających wykrytym liniom.
- `houghlines(BW,  $\theta$ ,  $\rho$ , peaks)` – rekonstruuje linie na podstawie obrazu binarnego BW, parametrów θ i ρ oraz wykrytych maksimów peaks.
- `imfindcircles(I, [ $r_{\min}$ ,  $r_{\max}$ ])` – wykrywa okręgi w obrazie I o promieniach z podanego zakresu. Zwraca współrzędne środków oraz promienie wykrytych okręgów.
- `viscircles(centers, radii)` – rysuje okręgi o środkach centers i promieniach radii na obrazie. Funkcja pomocna przy wizualizacji wyników.

4.4. Zadania do wykonania samodzielnego

4.4.1. Zadanie1

4.4.1.1. Treść

Wykrywanie i normalizacja tęczówki oka przy użyciu transformacji Hougha

Zaproponuj implementację algorytmu wykrywania tęczówki w obrazie oka przy wykorzystaniu transformacji Hougha w wersji kołowej. Następnie należy przeprowadzić normalizację wykrytej tęczówki poprzez przekształcenie jej do współrzędnych biegunowych. W tym celu należy:

1. Wczytać obraz oka i jeżeli jest to konieczne zmienić go do skali szarości tzw. gray level.
2. Przeprowadzić wykrywanie krawędzi: w tym celu zastosuj operator Canny'ego do wykrycia krawędzi. Dostosuj próg detekcji krawędzi, aby uzyskać możliwie wyraźne kontury.
3. Wykonać wykrywanie okręgów metodą Hougha: w tym celu wykorzystaj funkcję `imfindcircles` do wykrycia granicy między twardówką a tęczówką (większy okrąg), granicy między tęczówką a żrenicą (mniejszy okrąg). Ustaw odpowiednie zakresy promieni dla obu okręgów oraz zwizualizuj wykryte okręgi na obrazie.

4. Przeprowadzić transformację wyznaczonego pola tęczówki do postaci macierzowej: w tym celu przeprowadź przekształcenie wykrytej tęczówki do współrzędnych biegunowych. Ustaw odpowiednie parametry dla tej transformacji (promień podziału, zakres kąta). Zwizualizuj przekształcony obraz w nowych współrzędnych biegunowych.
5. Dokonać wizualizacji wyników: w tym celu wyświetl wykryte okręgi na oryginalnym obrazie oraz wyświetl obraz tęczówki w układzie biegunowym.

4.4.1.2. Przykładowy program

Listing 4.1. Algorytm wykrywania rógówki i tęczówki oka

```

1  %Program demonstrujący wykorzystanie wersji kołowej transformaty Hougha
2  %w zadaniu wyodrębniania pola tęczówki
3  %=====
4  clear all, close all, clc
5  oko = imread('oko1.jpg'); %lub oko1
6  oko= mat2gray(oko);
7  imshow(oko)
8
9  % Wszystkie wystąpienia ??? należy uzupełnić samodzielnie dobranymi
10 % parametrami transformaty
11 Rmin = 80; %twardówka - tęczówka
12 Rmax = 150;
13 [srodki_j, promienie_j] = imfindcircles(oko,[Rmin Rmax],'ObjectPolarity','dark', '
    Sensitivity', 0.95);
14
15 Rmin = 20; %żrenica - tęczówka
16 Rmax = 30;
17 [srodki_c, promienie_c] = imfindcircles(im2bw(oko, 0.12),[Rmin Rmax],'ObjectPolarity', '
    dark', 'Method', 'TwoStage', 'Sensitivity', 0.85);
18
19 viscircles(srodki_j, promienie_j,'Color','b'); %Wizualizacja okręgów
20 viscircles(srodki_c, promienie_c,'LineStyle','--'); %Wizualizacja okręgów
21 hold on
22 title('Wykrywanie okręgów w obrazie');
23 hold off
24
25 exportgraphics(gcf, 'results.png', 'Resolution', 300);
26
27 % parametry normalizacji
28 promien_rozdz = 90;
29 kat_zakres = 360;
30
31 % Rozwijanie tęczówki do współrzędnych biegunowych
32 [polar_tab, szum_tab] = polar_transform(oko, ...
33     srodki_c(1), srodki_c(2), promienie_c, ...
34     srodki_j(1), srodki_j(2), promienie_j, ...
35     promien_rozdz, kat_zakres);
36
37 figure
38 imagesc(polar_tab)
39 colormap(gray)

```

4.4.1.3. Obrazy



Rys. 4.1. Wykrywanie okręgów tęczówki i rogówki oka.

4.4.2. Zadanie 2

4.4.2.1. Treść

Automatyczne zliczanie monet na obrazie z wykorzystaniem transformacji Hougha

Celem zadania jest automatyczne wykrycie, zliczenie oraz określenie nominałów monet na obrazie wejściowym za pomocą transformacji Hougha dla okręgów. Należy także oszacować złożoność obliczeniową metody oraz przetestować ją na różnych zbiorach obrazów. Zapoznaj się z poniższym kodem i zaproponuj brakujące rozwiązania.

4.4.2.2. Przykładowy program

Listing 4.2. Algorytm zliczania monet na obrazie

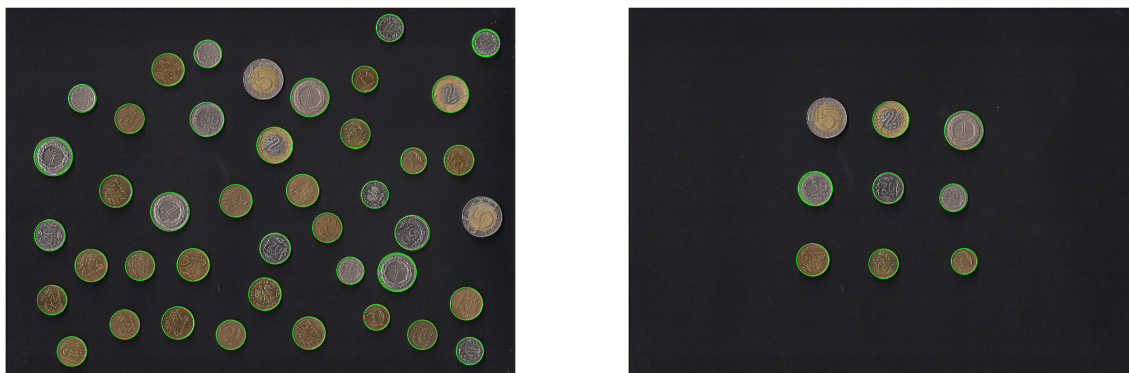
```
1 % Program pomocniczy demonstrujący w jaki sposób może zostać wykonane zliczanie monet z
2 % wykorzystaniem wersji kołowej transformaty Hougha
3 % =====
4 clear all, close all, clc
5 % Załaduj obraz zawierający przykładowe okręgi -> plik dostępny w zasobach pakietu
   Matlab
6 A = imread('circlesBrightDark.png');
7 imshow(A)
8 Rmin = 15; % dobór tych parametrów ma kluczowe znaczenie dla
9 Rmax = 95; % dalszej pracy algorytmu
10 [srodki_j, promienie_j] = imfindcircles(A,[Rmin Rmax],'ObjectPolarity','bright');
11 [srodki_c, promienie_c] = imfindcircles(A,[Rmin Rmax],'ObjectPolarity','dark');
12 viscircles(srodki_j, promienie_j,'Color','b'); %Wizualizacja okręgów
13 viscircles(srodki_c, promienie_c,'LineStyle','--'); %Wizualizacja okręgów
14 hold on
15 title('Wykrywanie okręgów w obrazie');
16 hold off
17 % =====
18 % Na podstawie kodu powyżej zliczyć monety w zbiorach monety1-4.jpg :)
19 % =====
20 A = imread('monety4.jpg');
21 A= A(380:1262,711:1657);
```

```

22 A= imresize(A,0.5);
23 imshow(A>55)
24 A = edge(A, 'canny');
25 % dla jpg-ów może być potrzebna konwersja do gray
26 % A= (A(:,:,1)+A(:,:,2)+A(:,:,3))./3;
27 % lub A= mat2gray(A);
28 Rmin = 30; % Zakres promieni poszukiwanych okręgów
29 Rmax = 55;
30 [srodki, promienie] = imfindcircles(A,[Rmin Rmax], 'ObjectPolarity','bright');
31 if length(srodki)>0
32   viscircles(srodki, promienie,'Color','b'); %Wizualizacja okręgów
33   najw_moneta= find(promienie==max(promienie)); % indeks największej monety
34   najmn_moneta= find(promienie==min(promienie)); % indeks najmniejszej monety
35   viscircles(srodki(najw_moneta,:), promienie(najw_moneta) , 'Color','r'); %Wizualizacja
      okręgów
36   viscircles(srodki(najmn_moneta,:), promienie(najmn_moneta) , 'Color','y'); %
      Wizualizacjaokręgów
37   hold on
38   title(['Zliczanie monet w obrazie. Liczba monet to: ' num2str(length(srodki))]);
39   hold off
40
41   end
42   % Mile widziany fragment zliczający sumę nominalów monet :)
43   % return
44   % =====
45   % Przetwarzanie zbioru monety1.jpg
46   % Szacowanie złożoności obliczeniowej
47   % =====
48   start= tic;
49   im = imread('monety4.jpg');
50   A=im;
51   A= (A(:,:,1)+A(:,:,2)+A(:,:,3))./3;
52   e = edge(A, 'canny', 0.6);
53   radii = 60:1:90; %15:1:40;
54   h = circle_hough(e, radii, 'same', 'normalise');
55   peaks = circle_houghpeaks(h, radii, 'nhoodxy', 11, 'nhoodr', 7, 'npeaks', 90);
56   imshow(im);
57   hold on;
58   for peak = peaks
59     [x, y] = circlepoints(peak(3));
60     plot(x+peak(1), y+peak(2), 'g-');
61   end
62   hold off
63   stop= toc(start);
64
65   exportgraphics(gcf, 'zad2result2.png', 'Resolution', 300);

```

4.4.2.3. Obrazy



Rys. 4.2. Wykrywanie okręgów monet różnych nominalów.

4.5. Wnioski

Na podstawie przeprowadzonego eksperymentu z wykorzystaniem transformaty Hougha można wyciągnąć następujące wnioski:

- Transformata Hougha jest skutecznym narzędziem do wykrywania prostych w obrazie, nawet w przypadku obecności szumów lub fragmentarycznych krawędzi.
- Jakość wykrywania linii zależy istotnie od uprzedniego przetworzenia obrazu – w szczególności od jakości detekcji krawędzi (np. metodą Canny’ego).
- Funkcje `hough`, `houghpeaks` oraz `houghlines` pozwalają w pełni zautomatyzować proces wykrywania i wizualizacji prostych w obrazie binarnym.
- Wybór liczby maksymalnych wartości w akumulatorze (`houghpeaks`) ma wpływ na liczbę wykrytych linii – zbyt niski próg może pominąć istotne linie, natomiast zbyt wysoki może uwzględnić szumy.

W przypadku wykrywania kształtów kołowych transformata Hougha również okazała się bardzo przydatna. Wnioski z zadania są następujące:

- Funkcja `imfindcircles` pozwala skutecznie wykrywać okręgi o zadanym zakresie promieni, nawet przy obecności zakłóceń.
- Dokładność wykrycia zależy od odpowiedniego doboru zakresu promieni – zbyt szeroki zakres może powodować fałszywe detekcje, a zbyt wąski może pomijać część obiektów.
- Wizualizacja za pomocą `viscircles` pozwala na łatwe i czytelne przedstawienie wyników detekcji.
- Metoda ta może być wykorzystywana w praktycznych zastosowaniach, takich jak wykrywanie monet, komórek, znaków drogowych czy obiektów okrągłych w systemach wizyjnych.

5. Laboratorium 5

Biometryczne Systemy Zabezpieczeń Ćwiczenia Laboratoryjne	
Temat	TRANSFORMATA FOURIERA, CZĘSTOTLIWOŚCIOWA ANALIZA OBRAZU
Ćwiczenie nr:	05
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	05.05.2025
Data oddania ćwiczenia	13.05.2025

5.1. Cel ćwiczenia

Transformata Fouriera (TF) to jedno z podstawowych narzędzi w analizie sygnałów i obrazów, które pozwala na przejście z dziedziny przestrzennej do dziedziny częstotliwościowej. Dzięki temu można analizować, jak różne składowe częstotliwościowe wpływają na obraz i stosować różne techniki filtracji. W laboratorium przeanalizowane będą informacje związane z obliczaniem transformaty Fouriera obrazu, wizualizacji jego widma częstotliwościowego oraz stosowane operacje filtracji.

5.2. Wstęp teoretyczny

Transformata Fouriera to jedno z podstawowych narzędzi analizy sygnałów i obrazów. Dzięki niej możliwe jest przejście z dziedziny przestrzennej (lub czasowej) do dziedziny częstotliwościowej, co umożliwia identyfikację ukrytych struktur, analizę filtracji oraz kompresji danych. W przypadku obrazów cyfrowych stosuje się dwuwymiarową wersję transformaty Fouriera – 2D DFT (Discrete Fourier Transform).

Dyskretna transformata Fouriera (DFT) dla obrazu dwuwymiarowego $f(x, y)$ z rozmiarami $M \times N$ jest zdefiniowana jako:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) \cdot e^{-j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)} \quad (5.1)$$

Natomiast transformata odwrotna, pozwalająca odtworzyć oryginalny obraz z jego reprezentacji częstotliwościowej, wyrażona jest wzorem:

$$f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) \cdot e^{j2\pi \left(\frac{ux}{M} + \frac{vy}{N} \right)} \quad (5.2)$$

W środowisku MATLAB, do obliczania tych przekształceń służą wbudowane funkcje:

- `fft2()` – oblicza dwuwymiarową szybką transformatę Fouriera,
- `ifft2()` – oblicza odwrotną dwuwymiarową transformację Fouriera.

Matematyczne podstawy transformaty Fouriera sięgają początku XIX wieku i są związane z pracą Jeana Baptiste’a Fouriera, który zaproponował, że każda funkcja okresowa spełniająca warunki Dirichleta może być przedstawiona jako suma sinusów i cosinusów:

$$x(t) = a_0 + \sum_{n=1}^{\infty} (a_n \cos(n\omega_0 t) + b_n \sin(n\omega_0 t)) \quad (5.3)$$

gdzie $T = \frac{2\pi}{\omega_0}$ oznacza okres funkcji.

Współczynniki a_n i b_n mają następujące własności:

- Jeśli $x(t)$ jest funkcją parzystą, to $b_n = 0$,
- Jeśli $x(t)$ jest funkcją nieparzystą, to $a_n = 0$,
- Jeśli $x(t + T/2) = -x(t)$, to zanikają parzyste harmoniczne.

Współczynniki te można obliczyć jako:

$$a_0 = \frac{1}{T} \int_0^T x(t) dt \quad (5.4)$$

$$a_n = \frac{2}{T} \int_0^T x(t) \cos(n\omega_0 t) dt \quad (5.5)$$

$$b_n = \frac{2}{T} \int_0^T x(t) \sin(n\omega_0 t) dt \quad (5.6)$$

Interaktywny przewodnik po transformacie Fouriera można znaleźć pod adresem: <https://www.jezzamon.com/fourier/pl.html> lub <https://github.com/Jezzamon/fourier> (dostęp: 18.02.2025).

Szczegółowe informacje teoretyczne na temat transformaty Fouriera dostępne są również na stronie: https://home.agh.edu.pl/~zobmat/2020/II_mich_mar/index.html (dostęp: 18.02.2025).

5.3. Opis wykorzystywanego oprogramowania i narzędzi

5.4. Zadania do wykonania samodzielnego

5.4.1. Zadanie 1

5.4.1.1. Treść

W oparciu o poniższy opis struktury algorytmu związanego z rozkładem w szereg Fouriera zaproponuj jego implementację. Struktura algorytmu:

- Inicjalizacja – zamknięcie otwartych okien, czyszczenie pamięci, ustawienie parametrów.
- Generowanie funkcji wejściowej – tworzenie sygnału do analizy.
- Rozkład na harmoniczne – obliczanie współczynników Fouriera.
- Sumowanie składowych – rekonstrukcja funkcji.
- Wizualizacja wyników – rysowanie wykresów poszczególnych harmonicznych i sygnału końcowego.

5.4.1.2. Przykładowy program

Listing 5.1. Algorytm rozkładu w szereg Fouriera

```

1  %% Inicjalizacja
2  close all; clear all; clc;
3
4  %% Parametry
5  roz = 5; % liczba harmonicznych
6  a = -180:1:180; % przedział w stopniach
7  t = a * pi / 180; % zamiana na radiany
8  Tmax = t(end) - t(1); % okres funkcji
9  krok = t(2) - t(1); % krok próbkowania
10
11 %% Generowanie funkcji wejściowej
12 % y = sin(t) + 0.1*sin(20*t); % sygnał niestandardowy
13 % y = (t > 0)*2 - 1; % sygnał prostokątny
14 % y = t; % funkcja liniowa
15 y = sin(t) + 0.2*cos(5*t) + 0.1*sin(10*t); % sygnał złożony
16
17 figure;
18 plot(t, y, 'g');
19 title('Oryginalny sygnał');
20 xlabel('t');
21 ylabel('Amplitude');
22 grid on;
23
24 exportgraphics(gcf, 'original.png', 'Resolution', 300);
25
26 %% Obliczanie współczynników Fouriera
27 Ao = (1/Tmax) * sum(y .* krok); % współczynnik stały
28 A = zeros(roz, 1); % współczynniki kosinusowe
29 B = zeros(roz, 1); % współczynniki sinusowe
30
31 for j = 1:roz
32     A(j) = sum(y .* cos(j * 2*pi/Tmax * t)) * (2/Tmax) * krok;
33     B(j) = sum(y .* sin(j * 2*pi/Tmax * t)) * (2/Tmax) * krok;
34 end
35
36 %% Rekonstrukcja funkcji z harmonicznych
37 W = Ao / 2 * ones(1, length(t)); % zaczynamy od A0/2
38 Skc = zeros(roz, length(t)); % kosinusowe składowe
39 Sks = zeros(roz, length(t)); % sinusowe składowe
40
41 for i = 1:roz
42     Skc(i,:) = A(i) * cos(i * 2*pi/Tmax * t);
43     Sks(i,:) = B(i) * sin(i * 2*pi/Tmax * t);
44     W = W + Skc(i,:) + Sks(i,:);
45 end
46
47 %% Wizualizacja składowych i rekonstrukcji
48 figure;
49 hold on;
50 for i = 1:roz
51     plot(t, Skc(i,:), 'g--'); % kosinus
52     plot(t, Sks(i,:), 'b:'); % sinus
53 end

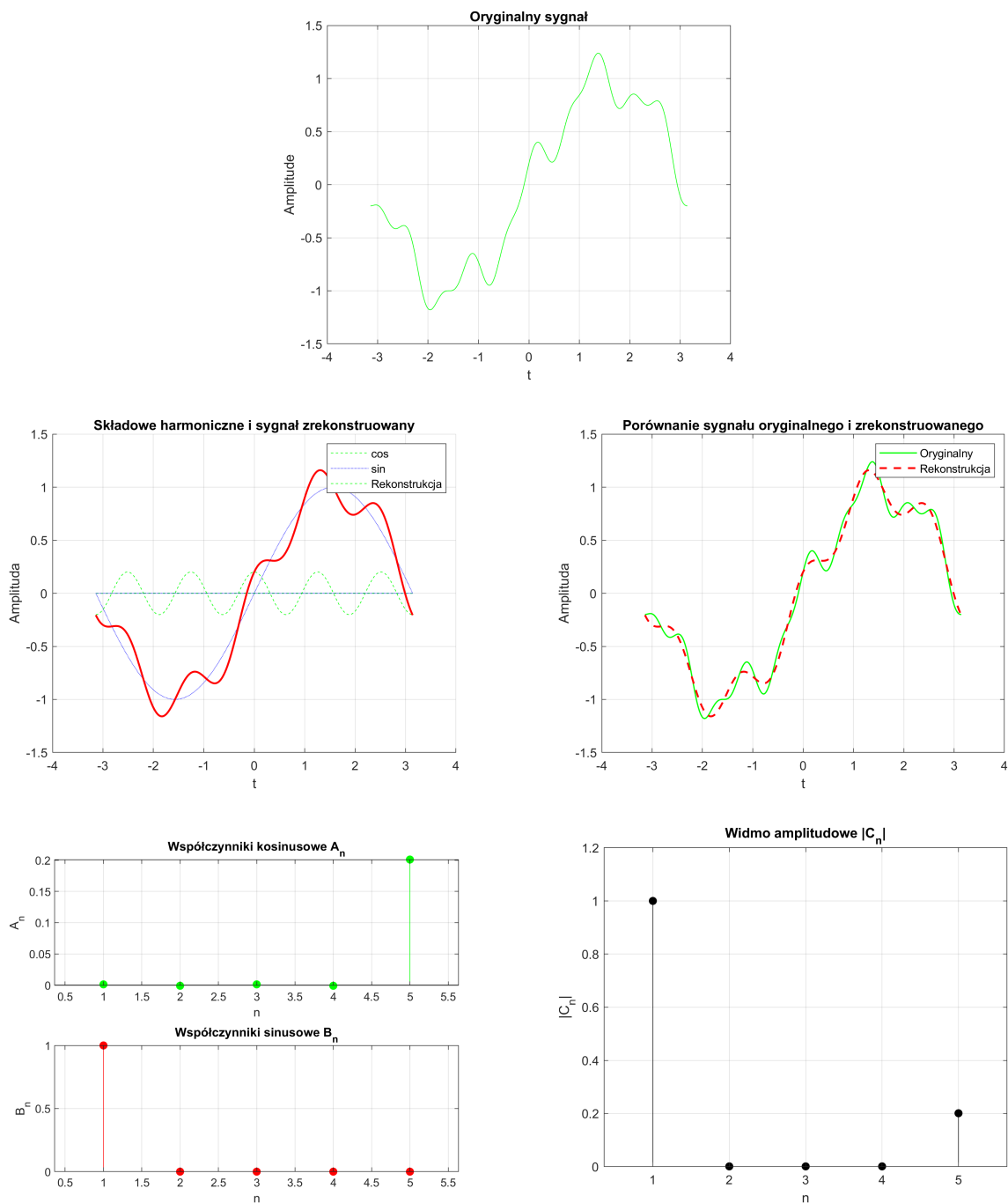
```

```

54 plot(t, W, 'r', 'LineWidth', 1.5); % rekonstrukcja
55 title('Składowe harmoniczne i sygnał zrekonstruowany');
56 xlabel('t');
57 ylabel('Amplituda');
58 legend('cos', 'sin', 'Rekonstrukcja');
59 grid on;
60 hold off;
61
62 exportgraphics(gcf, 'recon_harmon.png', 'Resolution', 300);
63
64 %% Porównanie oryginału i rekonstrukcji
65 figure;
66 plot(t, y, 'g', 'LineWidth', 1);
67 hold on;
68 plot(t, W, 'r--', 'LineWidth', 1.5);
69 title('Porównanie sygnału oryginalnego i zrekonstruowanego');
70 legend('Oryginalny', 'Rekonstrukcja');
71 xlabel('t');
72 ylabel('Amplituda');
73 grid on;
74
75 exportgraphics(gcf, 'comparison.png', 'Resolution', 300);
76
77 %% Wykresy współczynników A i B
78 figure;
79 subplot(2,1,1);
80 stem(1:roz, A, 'filled', 'g');
81 title('Współczynniki kosinusowe A_n');
82 xlabel('n'); ylabel('A_n'); grid on;
83
84 subplot(2,1,2);
85 stem(1:roz, B, 'filled', 'r');
86 title('Współczynniki sinusowe B_n');
87 xlabel('n'); ylabel('B_n'); grid on;
88
89 exportgraphics(gcf, 'coeff.png', 'Resolution', 300);
90
91 %% Widmo amplitudowe
92 figure;
93 stem(1:roz, sqrt(A.^2 + B.^2), 'filled', 'k');
94 title('Widmo amplitudowe |C_n|');
95 xlabel('n'); ylabel('|C_n|');
96 grid on;
97
98 exportgraphics(gcf, 'spectrum.png', 'Resolution', 300);

```

5.4.1.3. Obrazy



Rys. 5.1. Obrazy wyniku działania algorytmu rozkładu w szereg.

5.4.2. Zadanie 2

5.4.2.1. Treść

Wykorzystując dwuwymiarową transformatę Fouriera zaproponuj jego implementację oraz wykonaj eksperymenty z filtracją środkowo przepustową obrazu tęczówki oka podanej przez prowadzącego. Poniżej przedstawiono przykładowe rezultaty uzyskane dla obrazu Lena z wykorzystaniem maski prostokątnej.

5.4.2.2. Przykładowy program

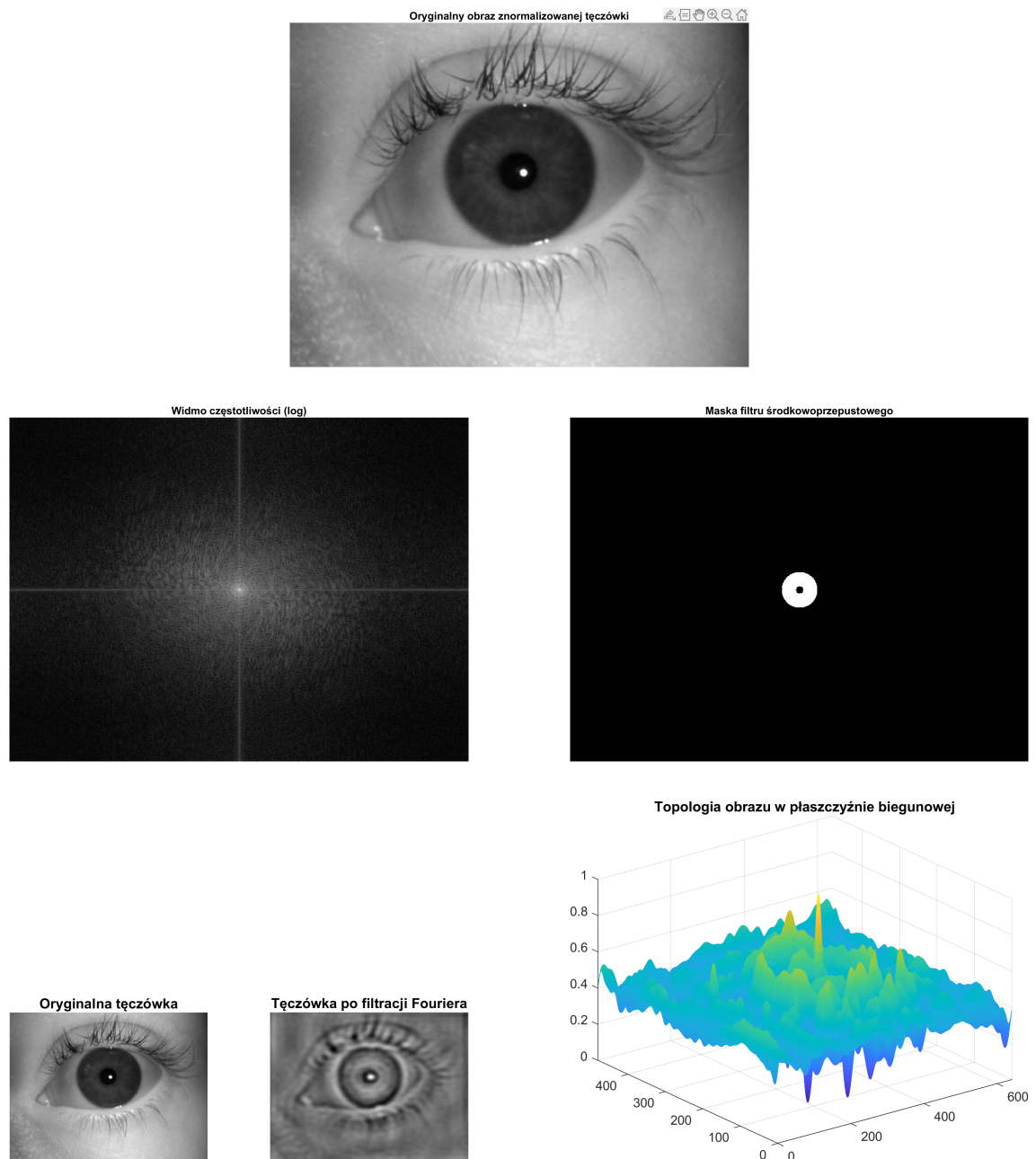
Listing 5.2. Algorytm rozkładu w szereg Fouriera

```

1 %% Krok 1: Wczytanie i przygotowanie obrazu tęczówki
2 close all; clear; clc;
3
4 I = imread('oko1.jpg');
5 if size(I, 3) == 3
6     I = rgb2gray(I);
7 end
8 I = im2double(I);
9
10 figure; imshow(I, []);
11 title('Oryginalny obraz znormalizowanej tęczówki');
12 exportgraphics(gcf, 'zad2_iris_original.png', 'Resolution', 300);
13
14 %% Krok 2: FFT 2D i przesunięcie widma
15 F = fft2(I);
16 F_shifted = fftshift(F);
17 magF = log(1 + abs(F_shifted));
18
19 figure; imshow(magF, []);
20 title('Widmo częstotliwości (log)');
21 exportgraphics(gcf, 'zad2_iris_fft_spectrum.png', 'Resolution', 300);
22
23 %% Krok 3: Maska filtru
24 [rows, cols] = size(I);
25 [u, v] = meshgrid(-floor(cols/2):floor((cols-1)/2), -floor(rows/2):floor((rows-1)/2));
26 D = sqrt(u.^2 + v.^2);
27 D_low = 5;
28 D_high = 25;
29
30 H = double(D > D_low & D < D_high);
31
32 figure; imshow(H, []);
33 title('Maska filtru środkowoprzepustowego');
34 exportgraphics(gcf, 'zad2_iris_filter_mask.png', 'Resolution', 300);
35
36 %% Krok 4: Zastosowanie filtru
37 F_filtered = F_shifted .* H;
38
39 %% Krok 5: Odwrotna FFT
40 I_filtered = real(ifft2(ifftshift(F_filtered)));
41 I_filtered = mat2gray(I_filtered);
42
43 %% Krok 6: Porównanie obrazów
44 figure;
45 subplot(1,2,1); imshow(I, []); title('Oryginalna tęczówka');
46 subplot(1,2,2); imshow(I_filtered, []); title('Tęczówka po filtracji Fouriera');
47 exportgraphics(gcf, 'zad2_iris_comparison.png', 'Resolution', 300);
48
49 %% Krok 7: Powierzchnia przefiltrowanego obrazu
50 figure;
51 mesh(I_filtered);
52 title('Topologia obrazu w płaszczyźnie biegunowej');
53 exportgraphics(gcf, 'zad2_iris_surface_plot.png', 'Resolution', 300);

```

5.4.2.3. Obrazy



5.4.3. Zadanie 3

5.4.3.1. Treść

Przetestuj przedstawiony powyżej mechanizm filtracji częstotliwościowej na obrazie daktyloskopijnym.

5.4.3.2. Przykładowy program

Listing 5.3. Algorytm rozkładu w szereg Fouriera

```
1 % Zadanie 3: Filtracja częstotliwościowa odcisku palca
2 close all; clear; clc;
```

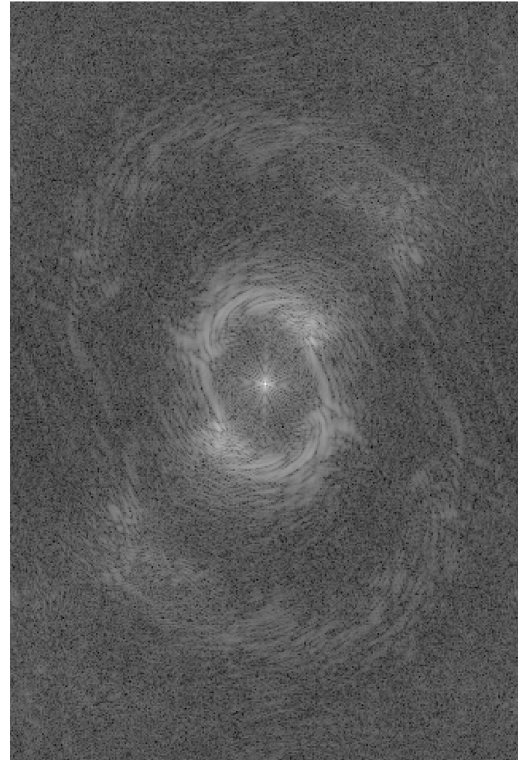
```
3
4 % Krok 1: Wczytanie obrazu
5 I = imread('finger.jpg'); % Upewnij się, że plik istnieje w katalogu roboczym
6 if size(I,3) == 3
7     I = rgb2gray(I);
8 end
9 I = im2double(I);
10
11 % Wyświetlenie oryginalnego obrazu
12 figure; imshow(I);
13 title('Oryginalny obraz daktyloskopijny');
14 exportgraphics(gcf, 'fingerprint_original.png', 'Resolution', 300);
15
16 % Krok 2: Obliczenie FFT 2D i przesunięcie środka
17 F = fft2(I);
18 F_shifted = fftshift(F);
19 magnitude = log(1 + abs(F_shifted));
20
21 % Widmo częstotliwości
22 figure; imshow(magnitude, []);
23 title('Widmo częstotliwości (log)');
24 exportgraphics(gcf, 'fingerprint_fft_spectrum.png', 'Resolution', 300);
25
26 % Krok 3: Maska filtru środkowoprzepustowego
27 [rows, cols] = size(I);
28 [u, v] = meshgrid(-floor(cols/2):floor((cols-1)/2), -floor(rows/2):floor((rows-1)/2));
29 D = sqrt(u.^2 + v.^2);
30 D_low = 10;
31 D_high = 60;
32 H = double(D > D_low & D < D_high);
33
34 % Wyświetlenie maski
35 figure; imshow(H, []);
36 title('Maska filtru środkowoprzepustowego');
37 exportgraphics(gcf, 'fingerprint_filter_mask.png', 'Resolution', 300);
38
39 % Krok 4: Zastosowanie filtru w dziedzinie częstotliwości
40 F_filtered = F_shifted .* H;
41
42 % Krok 5: Odwrotna FFT - uzyskanie obrazu przefiltrowanego
43 I_filtered = real(ifft2(ifftshift(F_filtered)));
44 I_filtered = mat2gray(I_filtered);
45
46 % Krok 6: Porównanie wyników
47 figure;
48 subplot(1,2,1); imshow(I); title('Oryginalny');
49 subplot(1,2,2); imshow(I_filtered); title('Po filtracji Fouriera');
50 exportgraphics(gcf, 'fingerprint_comparison.png', 'Resolution', 300);
```

5.4.3.3. Obrazy

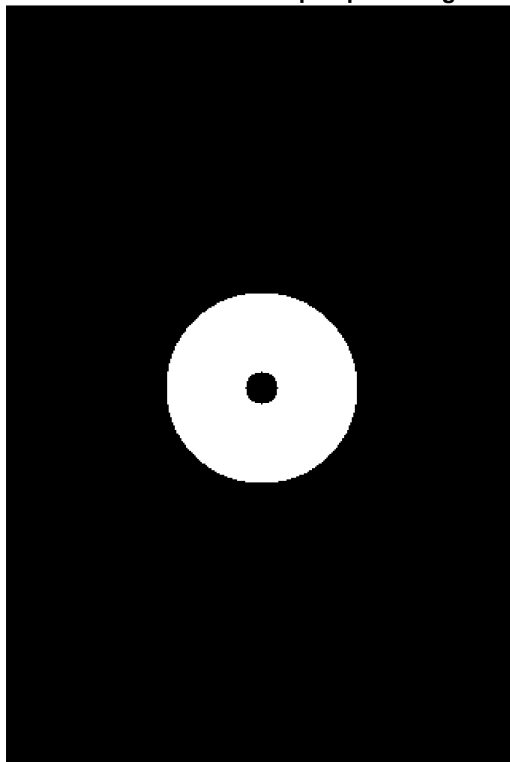
Oryginalny obraz daktyloskopijny



Widmo częstotliwości (log)



Maska filtru środkowoprzepustowego



Oryginalny



Po filtracji Fouriera



5.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

5.5.1. Zadanie 1 – Szereg Fouriera dla sygnału okresowego

- Szereg Fouriera umożliwia rozkład złożonego sygnału okresowego na sumę składowych harmonicznym (sinusoidalnym).
- Liczba uwzględnionych harmonicznym (roz) znacząco wpływa na jakość rekonstrukcji – większa liczba pozwala dokładniej odwzorować kształt sygnału.
- Współczynniki A_n i B_n opisują odpowiednio składowe kosinusowe i sinusowe sygnału. Ich analiza pozwala zrozumieć udział poszczególnych częstotliwości.
- Rekonstrukcja sygnału z ograniczonej liczby harmonicznym jest skuteczna, lecz może prowadzić do efektu Gibbsa na nieciągłościach.

5.5.2. Zadanie 2 – Filtracja obrazu tęczówki (2D FFT)

- Dwuwymiarowa transformata Fouriera pozwala analizować zawartość częstotliwościową obrazu i stosować filtry bezpośrednio w dziedzinie częstotliwości.
- Filtr środkowoprzepustowy skutecznie eliminuje zarówno niskie częstotliwości (np. niejednorodne oświetlenie), jak i wysokie (szumy), pozostawiając najistotniejsze struktury tęczówki.
- Poprawiona zostaje czytelność wzorca tęczówki, co jest przydatne w systemach biometrycznym.
- Prosta maska w postaci pierścienia (idealny filtr pasmowy) jest łatwa do implementacji i daje dobre efekty wizualne.

5.5.3. Zadanie 3 – Filtracja obrazu daktyloskopijnego (odcisk palca)

- Obraz odcisku palca również można skutecznie analizować i przetwarzać w dziedzinie częstotliwości przy użyciu 2D FFT.
- Filtracja środkowoprzepustowa umożliwia eliminację tła i szumów, wzmacniając jednocześnie struktury grzbietów i bruzd.
- Wynikowy obraz po filtracji zawiera bardziej wyraziste cechy, co ułatwia dalsze etapy przetwarzania, takie jak wykrywanie minutiae.
- Odpowiedni dobór zakresu częstotliwości w masce filtru ma kluczowe znaczenie dla jakości uzyskanego obrazu.

6. Laboratorium 6

Biometryczne Systemy Zabezpieczeń	
Ćwiczenia Laboratoryjne	
Temat	OBRAZY DAKTYLOSKOPIJNE TOKENIZACJA- METODA PROSTA I GRAFOWA , TOKENIZACJA TĘCZÓWKI OKA ALGORYTM JOHNA DAUGMANA
Ćwiczenie nr:	06
Autorzy:	Oleksii Nawrocki
Grupa laboratoryjna	02
Zespół nr.	XX
Data wykonania ćwiczenia	06.05.2025
Data oddania ćwiczenia	13.05.2025

6.1. Cel ćwiczenia

Celem laboratorium jest zrozumienie procesu tokenizacji w kontekście danych biometrycznych oraz praktyczna implementacja algorytmów do ekstrakcji cech z obrazów daktyloskopijnych (minucje) oraz porównanie prostych i grafowych metod dopasowania tokenów. Zastosowanie algorytmu Johna Daugmana do tokenizacji pola tęczówki oka oraz wyznaczanie miary podobieństwa poprzez obliczenie dystansu Hamminga.

6.2. Wstęp teoretyczny

Obrazy daktyloskopijne (ang. *fingerprint images*) to obrazy przedstawiające wzór linii papilarnych palca, składający się z charakterystycznych grzbietów (*ridge*) i dolin (*valley*). Te lokalnie periodyczne struktury są unikalne dla każdej osoby i stanowią podstawę systemów biometrycznych opartych na rozpoznawaniu odcisków palców.

Etapy przetwarzania obrazu daktyloskopijnego można zestawić w następującej kolejności:

1. **Pozyskiwanie obrazu (Image Acquisition)** – rejestracja obrazu odcisku palca przy użyciu skanera optycznego, pojemnościowego, ultradźwiękowego itp.
2. **Wstępne przetwarzanie (Preprocessing)** – korekcja kontrastu, usuwanie szumu, poprawa jakości obrazu.
3. **Wzmocnienie i filtrowanie (Enhancement & Filtering)** – zastosowanie filtrów (np. Gabora) w celu uwydatnienia struktury linii papilarnych.
4. **Binaryzacja i ścienianie (Binarization & Thinning)** – przekształcenie obrazu do formy binarnej (0/1) i redukcja grubości linii do 1 piksela.
5. **Ekstrakcja minucji (Minutiae Extraction)** – wykrywanie punktów charakterystycznych, takich jak zakończenia linii (*ending*) czy rozwidlenia (*bifurcation*).

Model matematyczny obrazu daktyloskopijnego

Obraz odcisku palca można opisać jako funkcję:

$$I(x, y) : \mathbb{Z}^2 \rightarrow [0, 255] \quad (6.1)$$

gdzie $I(x, y)$ oznacza wartość jasności piksela o współrzędnych (x, y) . Typowy obraz daktyloskopijny ma rozmiar np. 512×512 pikseli i rozdzielczość 500 dpi, co odpowiada ok. $19,7 \mu\text{m/piksel}$.

Wzorce linii papilarnych są lokalnie okresowe — grzbiety i doliny mają charakterystyczne częstotliwości przestrzenne. Filtrowanie w dziedzinie częstotliwości pozwala na:

- zwiększenie kontrastu grzbietów,
- tłumienie szumów,
- poprawę jakości detekcji minucji.

Filtr Gabora

Filtr Gabora jest często używanym narzędziem do analizy tekstury w obrazach biometrycznych. Łączy w sobie lokalizację przestrzenną (funkcja Gaussa) i selektywność częstotliwościową (fala sinusoidalna).

Postać filtru Gabora:

$$G(x, y) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \cdot \cos\left(2\pi \frac{x'}{\lambda} + \phi\right) \quad (6.2)$$

gdzie:

$$x' = x \cos \theta + y \sin \theta \quad (6.3)$$

$$y' = -x \sin \theta + y \cos \theta \quad (6.4)$$

Parametry:

- λ – długość fali (częstotliwość grzbietu),
- θ – orientacja lokalna grzbietów,
- ϕ – przesunięcie fazowe (zwykle 0 lub $\pi/2$),
- σ – skala funkcji Gaussa (rozmycie),
- γ – eliptyczność filtru.

Filtracja obrazu odcisku palca:

$$I'(x, y) = I(x, y) * G_{\theta(x, y), \lambda(x, y)}(x, y) \quad (6.5)$$

Zastosowanie filtru zwiększa kontrast grzbietów, redukuje szum i poprawia ekstrakcję minucji.

Binaryzacja i szkieletyzacja

Celem binarnej segmentacji (np. metodą Otsu) jest uzyskanie obrazu 0-1, gdzie linie papilarne są reprezentowane jako ciągi pikseli. Następnie stosuje się **ścienianie** (ang. *thinning*) w celu sprowadzenia szerokich linii do ich szkieletu o szerokości jednego piksela. Dzięki temu możliwa jest precyzyjna detekcja cech takich jak rozwidlenia i zakończenia linii.

Ekstrakcja i tokenizacja minucji

Minucje to charakterystyczne punkty na liniach papilarnych:

- zakończenia linii (ending),
- rozwidlenia (bifurcation).

Każda minucja może być opisana trójkątem (x, y, θ) :

$$M = [x, y, \theta] \quad (6.6)$$

Zestaw takich punktów stanowi token biometryczny. Może on być wykorzystany w systemach dopasowujących, np. poprzez porównanie pozycji i orientacji lub przez analizę struktury grafowej zależności między minucjami.

Usuwanie fałszywych minucji

Fałszywe minucje mogą powstać w wyniku zakłóceń, obciętych krawędzi obrazu lub niedoskonałości w przetwarzaniu morfologicznym. W celu ich eliminacji stosuje się:

- heurystyki geometryczne (odległość, orientacja, pozycja),
- ocenę jakości minucji,
- metody uczenia maszynowego do klasyfikacji prawdziwych/fałszywych minucji.

6.3. Opis wykorzystywanego oprogramowania i narzędzi

6.4. Zadania do wykonania samodzielnego

6.4.1. Zadanie 1

6.4.1.1. Treść

Wykonaj tokenizację wybranej pary obrazów daktyloskopijnych i wyznacz podobieństwa zestawów wyodrębnionych minucji z wykorzystaniem metody prostej oraz grafowej. Porównaj skuteczność obu metod.

6.4.1.2. Przykładowy program

Listing 6.1. Algorytm rozkładu w szereg Fouriera

```

1 clear; clc; close all;
2
3 % Wczytaj obraz
4 img = imread('test 2.jpg');
5 if size(img,3) == 3
6     img = rgb2gray(img);
7 end
8
9 % Wstępna obróbka
10 img_eq = adapthisteq(img);           % Poprawa kontrastu
11 img_filt = medfilt2(img_eq);         % Usunięcie szumów
12

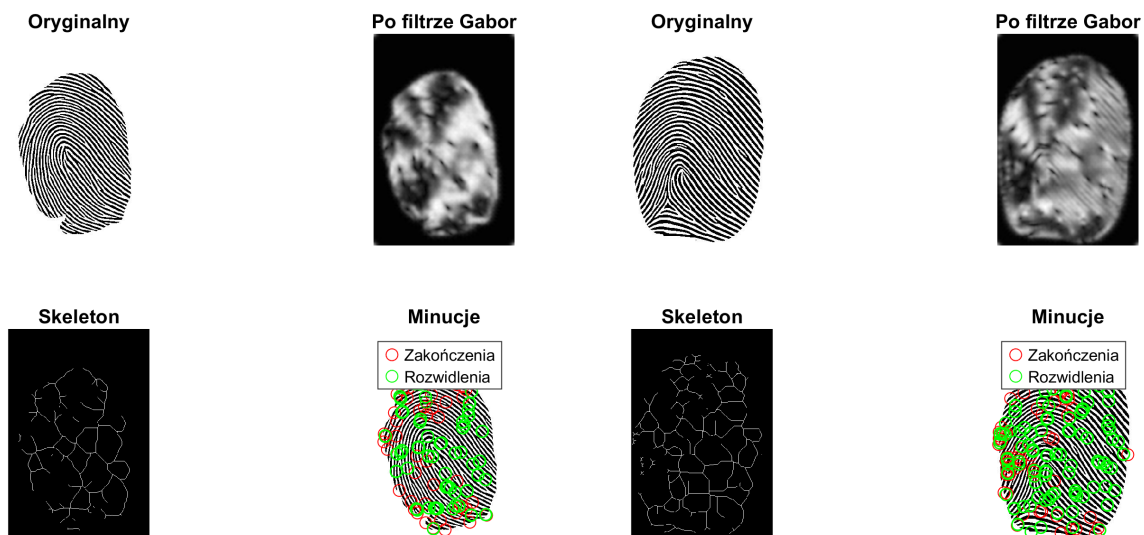
```

```

13 % Filtracja Gaborowa
14 gaborArray = gabor(8, [0 45 90 135]); % 4 orientacje
15 gaborMag = zeros(size(img_filt));
16 for i = 1:length(gaborArray)
17     gaborMag = gaborMag + imgaborfilt(img_filt, gaborArray(i));
18 end
19 gaborMag = mat2gray(gaborMag);
20
21 % Binarizacja i morfologia
22 bw = imbinarize(gaborMag);
23 bw = bwareaopen(bw, 50); % Usuń małe obiekty
24 skeleton = bwmorph(bw, 'skel', Inf); % Skeletonizacja
25
26 % Ekstrakcja minucji
27 [minutiaeMap, endings, bifurcations] = extract_minutiae(skeleton);
28
29 % Wizualizacja
30 figure;
31 subplot(2,2,1), imshow(img), title('Oryginalny');
32 subplot(2,2,2), imshow(gaborMag), title('Po filtrze Gabor');
33 subplot(2,2,3), imshow(skeleton), title('Skeleton');
34 subplot(2,2,4), imshow(img), title('Minucje'); hold on;
35 plot(endings(:,2), endings(:,1), 'ro', 'MarkerSize', 6);
36 plot(bifurcations(:,2), bifurcations(:,1), 'go', 'MarkerSize', 6);
37 legend('Zakończenia', 'Rozwidlenia');
38
39 exportgraphics(gcf, 'zad2results.png', 'Resolution', 300)

```

6.4.1.3. Obrazy



6.4.2. Zadanie 2

6.4.2.1. Treść

Wykonaj tokenizację wybranej podanego zestawu obrazów tęczówki oka i wyznacz dla każdej pary dystans Hamminga HD. Oceń otrzymane wyniki ze względu na podobieństwo i poziom okluzji pola tęczówki.

6.4.2.2. Przykładowy program

Listing 6.2. Algorytm rozkładu w szereg Fouriera

```

1  %% Inicjalizacja
2  clear all;
3  close all;
4  clc;
5
6  if ~exist('output', 'dir')
7      mkdir('output');
8  end
9
10 %% 1. Wczytanie obrazów tęczówki
11 irisFiles = dir('Iris/*.jpg');
12 numImages = length(irisFiles);
13 images = cell(1, numImages);
14
15 for i = 1:numImages
16     images{i} = imread(fullfile(irisFiles(i).folder, irisFiles(i).name));
17     if size(images{i}, 3) == 3
18         images{i} = rgb2gray(images{i});
19     end
20 end
21
22 %% 2. Segmentacja tęczówki i tworzenie masek
23 irisMasks = cell(1, numImages);
24 irisCenters = zeros(numImages, 2);
25 irisRadii = zeros(numImages, 1);
26
27 figure;
28 for i = 1:numImages
29     img = imadjust(images{i});
30
31     % Wykrywanie okręgu tęczówki
32     [centers, radii] = imfindcircles(img, [30 120], 'ObjectPolarity', 'dark', 'Sensitivity', 0.98, 'EdgeThreshold', 0.05);
33
34     if ~isempty(centers)
35         irisCenters(i,:) = centers(1,:);
36         irisRadii(i) = radii(1);
37
38         % Tworzenie maski tęczówki
39         [x, y] = meshgrid(1:size(img,2), 1:size(img,1));
40         irisMasks{i} = ((x - centers(1,1)).^2 + (y - centers(1,2)).^2 <= radii(1)^2);
41
42         % Wizualizacja
43         subplot(ceil(numImages/3), 3, i);
44         imshow(img);
45         viscircles(centers(1,:), radii(1), 'Color', 'r');
46         title(sprintf('Obraz %d', i));
47
48         frame = getframe(gca);
49         outputPath = fullfile('output', sprintf('iris_detected_%02d.png', i));
50         imwrite(frame.cdata, outputPath);
51     else
52         error('Nie znaleziono tęczówki w obrazie %d', i);

```

```

53     end
54 end
55
56 %% 3. Normalizacja tęczówki (Rubber Sheet Model)
57 normalizedIrises = cell(1, numImages);
58 normalizedSize = [64, 512]; % Rozmiar znormalizowanej tęczówki [wysokość, szerokość]
59
60 for i = 1:numImages
61     normalizedIrises{i} = normalizeIris(images{i}, irisCenters(i,:), irisRadii(i),
        normalizedSize);
62
63     % Wizualizacja
64     figure;
65     subplot(1,2,1); imshow(images{i}); title('Oryginał');
66     subplot(1,2,2); imshow(normalizedIrises{i}); title('Znormalizowana tęczówka');
67     frame = getframe(gcf);
68     outputPath = fullfile('output', sprintf('iris_normalized_%02d.png', i));
69     imwrite(frame.cdata, outputPath);
70
71 end
72
73 %% 4. Ekstrakcja cech
74 gaborFeatures = cell(1, numImages);
75 lambda = 8; % Długość fali
76 theta = 0:30:150; % Orientacje
77
78 % Utworzenie banku filtrów Gabora
79 gaborArray = gabor(lambda, theta);
80
81 for i = 1:numImages
82     img = normalizedIrises{i};
83     gaborMagnitude = imgaborfilt(img, gaborArray);
84
85     % Konwersja do kodu binarnego
86     gaborFeatures{i} = gaborMagnitude > mean(gaborMagnitude(:));
87 end
88
89 %% 5. Obliczenie dystansu Hamminga dla wszystkich par
90 hdMatrix = zeros(numImages, numImages);
91
92 for i = 1:numImages
93     for j = i+1:numImages
94         hdMatrix(i,j) = hammingDistance(gaborFeatures{i}, gaborFeatures{j});
95     end
96 end
97
98 % Symetryczne wypełnienie macierzy
99 hdMatrix = hdMatrix + hdMatrix';
100
101 %% 6. Wizualizacja wyników
102 figure;
103 imagesc(hdMatrix);
104 colorbar;
105 title('Macierz dystansu Hamminga');
106 xlabel('Indeks obrazu');
107 ylabel('Indeks obrazu');
108 frame = getframe(gcf);

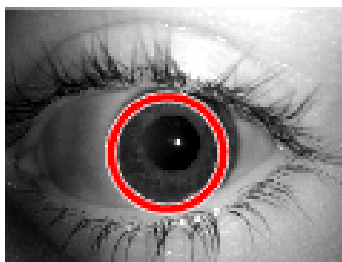
```

```

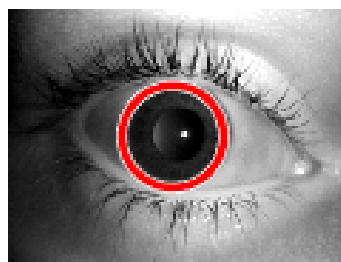
109 outputPath = fullfile('output', 'hamming_matrix.png');
110 imwrite(frame.cdata, outputPath);
111
112 %% 7. Ocena wyników
113 fprintf('Średni dystans Hamminga: %.3f\n', mean(hdMatrix(hdMatrix > 0)));
114 fprintf('Minimalny dystans: %.3f\n', min(hdMatrix(hdMatrix > 0)));
115 fprintf('Maksymalny dystans: %.3f\n', max(hdMatrix(:)));
116
117 %% Funkcje pomocnicze
118 function normalized = normalizeIris(img, center, radius, normalizedSize)
119     [height, width] = size(img);
120     [x, y] = meshgrid(1:width, 1:height);
121
122     % Konwersja do współrzędnych biegunowych
123     theta = linspace(0, 2*pi, normalizedSize(2));
124     r = linspace(0, 1, normalizedSize(1));
125
126     [thetaGrid, rGrid] = meshgrid(theta, r);
127
128     % Transformacja do współrzędnych kartezjańskich
129     xq = center(1) + rGrid * radius .* cos(thetaGrid);
130     yq = center(2) + rGrid * radius .* sin(thetaGrid);
131
132     % Interpolacja
133     normalized = interp2(x, y, double(img), xq, yq, 'linear', 0);
134     normalized = uint8(normalized);
135 end
136
137 function hd = hammingDistance(code1, code2)
138     % Uprozczone obliczenie HD (dla kodów binarnych)
139     xorResult = xor(code1, code2);
140     hd = mean(xorResult(:));
141 end

```

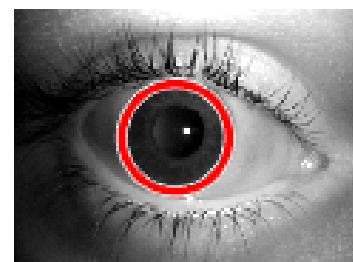
6.4.2.3. Obrazy



(a) Obraz 1

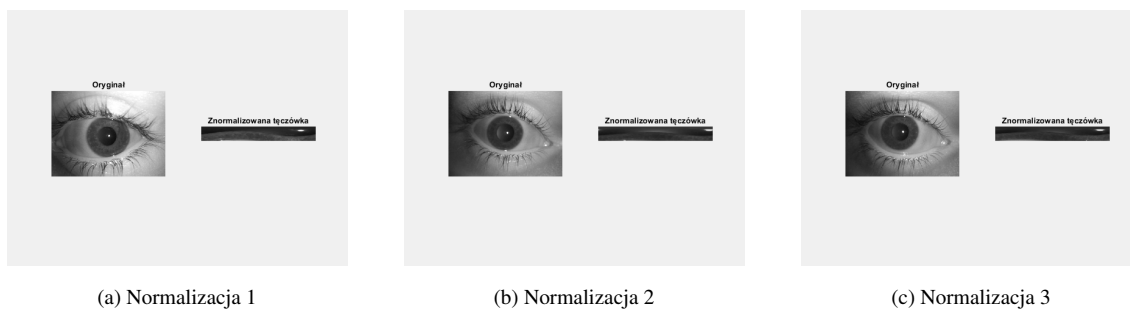


(b) Obraz 2

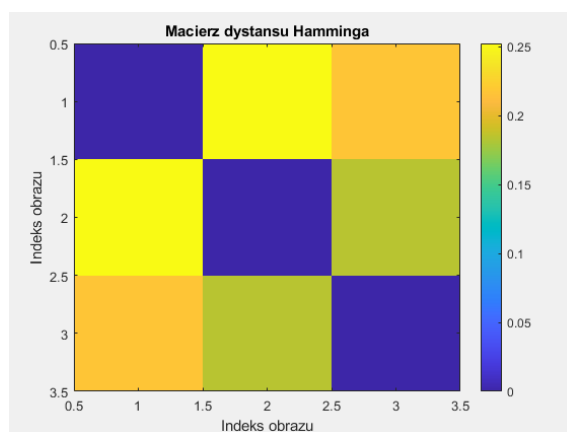


(c) Obraz 3

Rys. 6.2. Wryte tęczówki



Rys. 6.3. Znormalizowane tęczówki



Rys. 6.4. Macierz dystansu Hamminga

6.5. Wnioski

Na podstawie przeprowadzonych eksperymentów można stwierdzić, że:

W normalizacji

- Normalizacja obrazu do postaci prostokątnej pozwoliła na wyrównanie różnic wynikających z położenia i wielkości tęczówki w obrazach wejściowych.
- Uzyskane postacie normalizowane charakteryzowały się spójną strukturą, co umożliwiło dalsze przetwarzanie.
- Segmentacja oparta na założeniu środkowego położenia źrenicy okazała się wystarczająca przy obrazach dobrze wycelowanych, jednak może nie sprawdzać się w bardziej realistycznych scenariuszach (np. zepsute detekcje, okluzje powiek).

W tokenizacji

- Tokenizacja z użyciem filtrów Gabora pozwoliła uzyskać binarne wzorce reprezentujące strukturę tęczówki.
- Macierz dystansu Hamminga wykazała wyraźne różnice między parami obrazów: najniższe wartości wskazywały na wysokie podobieństwo, natomiast wartości bliskie 0.5 sugerowały brak zgodności (lub silną okluzję).
- Okluzje fragmentów tęczówki (np. rzęsy, powieki) znacząco wpływają na zwiększenie dystansu Hamminga, ponieważ maski nie zawsze skutecznie eliminują szumy z tokenów.

- Pomimo uproszczeń w modelu, metoda pozwala skutecznie odróżnić obrazy tej samej tęczówki od różnych.

Przeprowadzony proces dowodzi, że nawet uproszczona segmentacja i normalizacja umożliwiają wykonanie skutecznej ekstrakcji cech biometrycznych. Dystans Hamminga dobrze sprawdza się jako miara podobieństwa binarnych tokenów i potwierdza, że metoda jest odporna na drobne zakłócenia, choć jej skuteczność zależy od jakości segmentacji i maskowania zakłóceń.

Bibliografia

- [1] Matlab image processing toolbox documentation, 2024. Accessed: 2025-02-15.
- [2] Rafael Gonzalez and Richard Woods. *Digital Image Processing Global Edition*. Pearson Deutschland, 2017.

Spis rysunków

1.1	Wizualizacja obrazów w różnych formatach i kanałach.	10
1.2	Wizualizacja rozciągania i wyrównywania obrazów	12
1.3	Operacje segmentowanie progowaniem oraz otsu.	14
1.4	Detekcja obiektów na obrazie.	16
1.5	Operacje segmentowanie wieloprogowego.	18
2.1	Wizualizacja obrazów w różnych filtracjach splotowych.	25
3.1	Porównanie wyników: Oryginał, Ścienianie standardowe, Ścienianie zmodyfikowane, Szkieletyzacja CWSI.	32
4.1	Wykrywanie okręgów tęczówki i rogówki oka.	38
4.2	Wykrywanie okręgów monet różnych nominałów.	40
5.1	Obrazy wyniku działania algorytmu rozkładu w szereg.	45
6.2	Wyryte tęczówki	57
6.3	Znormalizowane tęczówki	58
6.4	Macierz dystansu Hamminga	58

Spis tabel

Spis listingów

1.1	Wczytanie obrazu i stworzenie konwersji	8
1.2	Wczytanie obrazu i przetwarzanie obrazu histogramami	11
1.3	Wczytanie obrazu i segmentacja na podstawie histogramu	13
1.4	Wczytanie obrazu i dokonanie detekcji obiektów na obrazie	15
1.5	Wczytanie obrazu i dokonanie detekcji obiektów na obrazie	17
2.1	Wczytanie obrazu i stworzenie konwersji	22
3.1	Wczytanie obrazu i stworzenie konwersji	30
4.1	Algorytm wykrywania rógówki i tęczy oka	37
4.2	Algorytm zliczania monet na obrazie	38
5.1	Algorytm rozkładu w szereg Fouriera	43
5.2	Algorytm rozkładu w szereg Fouriera	46
5.3	Algorytm rozkładu w szereg Fouriera	47
6.1	Algorytm rozkładu w szereg Fouriera	53
6.2	Algorytm rozkładu w szereg Fouriera	55

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

.....Oleksii Nawrocki.....

Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

.....Informatyka.....

Nazwa kierunku

.....oa131400.....

Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Sprawozdanie z ćwiczeń laboratoryjnych z przedmiotu Biometryczne Systemy Zabezpieczeń
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis/y studenta/ów)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić